

Part I: Foundational Dart Programs —

Program 1(a): Data Types & Variables Demonstration

Objective:

To write a Dart program that demonstrates the fundamental primitive data types: `String`, `int`, `double`, and `bool`.

Theory & Procedure:

1. **Theory:** Dart is a strongly typed language with type inference. Standard built-in primitive types include `int` (64-bit integers), `double` (64-bit floating-point numbers), `String` (UTF-16 code units), and `bool` (boolean values `true` or `false`).
2. **Procedure:** Declare explicit variables for each data type, assign valid values, and display them using string interpolation (`$variable`).

// Program 1(a): Data Types & Declarations

```
void main() {  
  // Variable Declarations  
  String studentName = "Swarnandhra";  
  int rollNumber = 101;  
  double gpa = 8.75;  
  bool isEnrolled = true;  
  
  // Output using String Interpolation  
  print('--- Student Profile ---');  
  print('Name      : $studentName');  
  print('Roll No   : $rollNumber');  
  print('GPA       : $gpa');  
  print('Active    : $isEnrolled');  
}
```

Sample Output:

```
--- Student Profile ---  
Name      : Swarnandhra  
Roll No   : 101  
GPA       : 8.75  
Active    : true
```

Program 1(b): Student Total & Average Calculation

Objective:

To compute the total marks and average score for a student across three subjects.

Theory & Procedure:

1. **Theory:** Arithmetic computation in Dart evaluates numeric expressions using arithmetic operators. Division of integers results in a `double`.
2. **Procedure:** Store three integer subject marks, sum them into a `total` variable, divide by `3.0` to calculate the `average`, and format the average using `.toStringAsFixed(2)`.

// Program 1(b): Student Total & Average Calculation

```
void main() {  
  String studentName = "Aravind";  
  int mark1 = 85;  
  int mark2 = 92;  
  int mark3 = 78;  
  
  int total = mark1 + mark2 + mark3;  
  double average = total / 3;  
  
  print('--- Grade Report ---');  
  print('Student Name : $studentName');  
  print('Subject Marks : $mark1, $mark2, $mark3');  
  print('Total Marks : $total / 300');  
  print('Average Marks : ${average.toStringAsFixed(2)}%');  
}
```

Sample Output:

```
-- Grade Report ---  
Student Name : Aravind  
Subject Marks : 85, 92, 78  
Total Marks : 255 / 300  
Average Marks : 85.00%
```

Program 2(a): Standard Arithmetic Operators

Objective:

To evaluate all binary arithmetic operators in Dart including integer truncation division and modulo operations.

Theory & Procedure:

1. **Theory:** Dart provides basic arithmetic operators: addition (`+`), subtraction (`-`), multiplication (`*`), standard division (`/`), integer division (`~/`), and modulo remainder (`%`).
2. **Procedure:** Declare two integer operands and print the result of applying each arithmetic operator.

// Program 2(a): Standard Arithmetic Operators

```
void main() {  
  int num1 = 25;  
  int num2 = 4;
```

```

print('Operands: num1 = $num1, num2 = $num2\n');
print('Addition (num1 + num2)      : ${num1 + num2}');
print('Subtraction (num1 - num2)   : ${num1 - num2}');
print('Multiplication (num1 * num2) : ${num1 * num2}');
print('Standard Division (num1 / num2): ${num1 / num2}');
print('Truncating Div (num1 ~/ num2) : ${num1 ~/ num2}');
print('Modulo Remainder (num1 % num2) : ${num1 % num2}');
}

```

Sample Output:

Operands: num1 = 25, num2 = 4

```

Addition (num1 + num2)      : 29
Subtraction (num1 - num2)   : 21
Multiplication (num1 * num2) : 100
Standard Division (num1 / num2): 6.25
Truncating Div (num1 ~/ num2) : 6
Modulo Remainder (num1 % num2) : 1

```

Program 2(b): Rectangle Area & Perimeter Calculation

Objective:

To compute the area and perimeter of a rectangle given its length and width.

Theory & Procedure:

1. **Theory:** Area=length×width and Perimeter=2×(length+width)
2. **Procedure:** Accept double values for length and width, execute the geometric equations, and print the formatted outputs.

// Program 2(b): Rectangle Area & Perimeter

```

void main() {
    double length = 12.5;
    double width = 5.0;

    double area = length * width;
    double perimeter = 2 * (length + width);

    print('Rectangle Dimensions: $length x $width');
    print('Area          : $area sq units');
    print('Perimeter       : $perimeter units');
}

```

Sample Output:

```

Rectangle Dimensions: 12.5 x 5.0
Area          : 62.5 sq units
Perimeter       : 35.0 units

```

Program 3(a): Number Sign Evaluation

Objective:

To classify a given numerical value as Positive, Negative, or Zero using multi-branch conditional statements.

Theory & Procedure:

1. **Theory:** `if-else-if` constructs allow sequential evaluation of boolean conditions until one evaluates to `true`.
2. **Procedure:** Check if `number > 0`. If true, classify as positive; else if `number < 0`, classify as negative; otherwise, zero.

// Program 3(a): Positive / Negative / Zero Check

```
void checkNumberSign(int number) {  
    if (number > 0) {  
        print('Number $number is POSITIVE.');    } else if (number < 0) {  
        print('Number $number is NEGATIVE.');    } else {  
        print('Number $number is ZERO.');    }  
}
```

```
void main() {  
    checkNumberSign(15);  
    checkNumberSign(-8);  
    checkNumberSign(0);  
}
```

Sample Output:

Number 15 is POSITIVE.
Number -8 is NEGATIVE.
Number 0 is ZERO.

Program 3(b): Grade Evaluation System

Objective:

To assign a letter grade based on percentage marks using `if-else-if` conditional logic.

Theory & Procedure:

1. **Theory:** Multi-threshold grading evaluates continuous ranges using comparative bounds.
2. **Procedure:** Distinction=marks $\geq$ 85, First Class=marks $\geq$ 70, Second Class=marks $\geq$ 50, Pass=marks $\geq$ 40, Fail=marks $<$ 40.

// Program 3(b): Grade Evaluation System

```

void printGrade(double percentage) {
    String grade;

    if (percentage >= 85) {
        grade = 'Distinction';
    } else if (percentage >= 70) {
        grade = 'First Class';
    } else if (percentage >= 50) {
        grade = 'Second Class';
    } else if (percentage >= 40) {
        grade = 'Pass';
    } else {
        grade = 'Fail';
    }

    print('Marks Percentage: ${percentage}% -> Grade: $grade');
}

void main() {
    printGrade(88.5);
    printGrade(72.0);
    printGrade(42.5);
    printGrade(31.0);
}

```

Sample Output:

```

Marks Percentage: 88.5% -> Grade: Distinction
Marks Percentage: 72.0% -> Grade: First Class
Marks Percentage: 42.5% -> Grade: Pass
Marks Percentage: 31.0% -> Grade: Fail

```

Program 4(a): Multiplication Table Generation

Objective:

To display the multiplication table for a specified integer using a **for** loop.

Theory & Procedure:

1. **Theory:** A fixed-count **for** loop iterates over a index variable from \$1\$ to \$N\$.
2. **Procedure:** Loop \$i\$ from 1 to 10 and print \$N \times i = (N \cdot i)\$ on each line.

// Program 4(a): Multiplication Table

```

void generateTable(int number) {
    print('--- Multiplication Table for $number ---');
    for (int i = 1; i <= 10; i++) {
        print('$number x $i = ${number * i}');
    }
    print("");
}

```

```

}

void main() {
    generateTable(7);
}

```

Sample Output:

--- Multiplication Table for 7 ---

```

7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70

```

Program 4(b): Sum of First N Natural Numbers

Objective:

To compute the sum of numbers from 1 to N using a **while** loop.

Theory & Procedure:

1. **Theory:** Iterative accumulation uses a loop counter and a running sum accumulator.
2. **Procedure:** Initialize **sum = 0** and **counter = 1**. Increment **counter** while **counter <= n**, adding **counter** to **sum**.

// Program 4(b): Sum of First N Natural Numbers

```

void main() {
    int n = 50;
    int sum = 0;
    int counter = 1;

    while (counter <= n) {
        sum += counter;
        counter++;
    }

    print('The sum of first $n natural numbers is: $sum');
}

```

Sample Output:

The sum of first 50 natural numbers is: 1275

Program 5(a): List Extreme Values (Max/Min)

Objective:

To search an unsorted `List<int>` for its maximum and minimum elements.

Theory & Procedure:

1. **Theory:** Direct iteration over linear list collections evaluates relative magnitude against stored extreme markers.
2. **Procedure:** Set `max` and `min` to the first list element `numbers[0]`. Iterate through the remaining elements updating `max` and `min` conditionally.

// Program 5(a): Find Largest & Smallest Element in a List

```
void main() {  
    List<int> numbers = [34, 12, 89, 5, 67, 92, 23];  
  
    if (numbers.isEmpty) {  
        print('List is empty.');        return;  
    }  
  
    int smallest = numbers[0];  
    int largest = numbers[0];  
  
    for (int num in numbers) {  
        if (num < smallest) smallest = num;  
        if (num > largest) largest = num;  
    }  
  
    print('Input List : $numbers');  
    print('Smallest  : $smallest');  
    print('Largest   : $largest');  
}
```

Sample Output:

```
Input List : [34, 12, 89, 5, 67, 92, 23]  
Smallest  : 5  
Largest   : 92
```

Program 5(b): Map Filtering by Threshold

Objective:

To store student names and marks in a `Map<String, int>` and filter entries where `marks > 75`.

Theory & Procedure:

1. **Theory:** A `Map` stores key-value pairs. Iteration using `.forEach()` or conditional transformations (`.where()`) allows filtering based on value thresholds.

2. **Procedure:** Populate a map, iterate using `.forEach()`, and print key-value pairs matching the constraint `value > 75`.

// Program 5(b): Map Filtering (Marks > 75)

```
void main() {  
    Map<String, int> studentMarks = {  
        'Alice': 82,  
        'Bob': 64,  
        'Charlie': 91,  
        'Diana': 73,  
        'Ethan': 78,  
    };  
  
    print('Complete Record Map:');  
    print(studentMarks);  
  
    print("\nStudents scoring above 75 (Distinction/High Achievement):");  
    studentMarks.forEach((name, mark) {  
        if (mark > 75) {  
            print('- $name: $mark marks');  
        }  
    });  
}
```

Sample Output:

Complete Record Map:

{Alice: 82, Bob: 64, Charlie: 91, Diana: 73, Ethan: 78}

Students scoring above 75 (Distinction/High Achievement):

- Alice: 82 marks
- Charlie: 91 marks
- Ethan: 78 marks

Program 6(a): Prime Number Check Function

Objective:

To write a reusable function that checks whether a given positive integer is a prime number.

Theory & Procedure:

1. **Theory:** A prime number is a natural number greater than 1 that has no positive divisors other than 1 and itself. A number n can be tested for primality by checking divisibility up to $\sqrt{n}$.
2. **Procedure:** Define a function `bool isPrime(int n)`. If $n \leq 1$, return `false`. Loop i from 2 up to $\sqrt{n}$; if $n \% i == 0$, return `false`. If no divisors are found, return `true`.

// Program 6(a): Prime Number Check Function

```

bool isPrime(int n) {
    if (n <= 1) return false;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) return false;
    }
    return true;
}

void main() {
    List<int> testNumbers = [1, 2, 13, 20, 29, 33];

    print('--- Prime Number Evaluation ---');
    for (int number in testNumbers) {
        if (isPrime(number)) {
            print('$number is a PRIME number.');
```

Sample Output:

```

--- Prime Number Evaluation ---
1 is NOT a prime number.
2 is a PRIME number.
13 is a PRIME number.
20 is NOT a prime number.
29 is a PRIME number.
33 is NOT a prime number.
```

Program 6(b): Factorial Computation Function

Objective:

To calculate the factorial of an integer using a recursive function.

Theory & Procedure:

1. **Theory:** The factorial of a non-negative integer n (denoted as $n!$) is the product of all positive integers less than or equal to n . Base cases: $0! = 1$ and $1! = 1$.
2. **Procedure:** Define `int factorial(int n)`. If $n \leq 1$, return 1; otherwise, return $n \times \text{factorial}(n - 1)$.

// Program 6(b): Factorial Computation

```

int factorial(int n) {
    if (n < 0) {
        throw ArgumentError('Factorial is not defined for negative numbers.');
```

```

    if (n <= 1) return 1;
    return n * factorial(n - 1);
}

void main() {
    List<int> inputs = [0, 5, 7];

    print('--- Factorial Computations ---');
    for (int val in inputs) {
        print('Factorial of $val ($val!) = ${factorial(val)}');
    }
}

```

Sample Output:

```

--- Factorial Computations ---
Factorial of 0 (0!) = 1
Factorial of 5 (5!) = 120
Factorial of 7 (7!) = 5040

```

Program 7(a): Vowel Count in String

Objective:

To iterate through a string input and count the number of vowels present.

Theory & Procedure:

1. **Theory:** Strings in Dart are sequences of UTF-16 code units. Text can be converted to a uniform case (lowercase) and evaluated against a set of vowel characters (a, e, i, o, u).
- 2.
3. **Procedure:** Accept a string, convert it using `.toLowerCase()`, initialize `vowelCount = 0`, iterate through each character, and increment the count if it matches any vowel.

// Program 7(a): Count Vowels in a String

```

int countVowels(String text) {
    int count = 0;
    String vowels = 'aeiou';
    String normalized = text.toLowerCase();

    for (int i = 0; i < normalized.length; i++) {
        if (vowels.contains(normalized[i])) {
            count++;
        }
    }
    return count;
}

void main() {

```

```
String sentence = "Swarnandhra College of Engineering";
int totalVowels = countVowels(sentence);

print('Input Text   : "$sentence"');
print('Total Vowels : $totalVowels');
}
```

Sample Output:

```
Input Text   : "Swarnandhra College of Engineering"
Total Vowels : 12
```

Program 7(b): Sound Null Safety Operators

Objective:

To demonstrate Dart's sound null safety system including nullable types (`?`), safe call (`?.`), null-coalescing (`??`), null assertion (`!`), and `late` initialization.

Theory & Procedure:

1. **Theory:** Dart prevents null dereference errors at compile time by distinguishing nullable (`Type?`) from non-nullable (`Type`) types.
2. **Procedure:** Declare a nullable string, use `?.length` for safe access, `?? 'Default'` to provide fallbacks, `late` for deferred initialization, and `!` to assert non-nullability.

// Program 7(b): Sound Null Safety Operators

```
void main() {
  // 1. Nullable variable declaration
  String? nullableName;

  // 2. Null-coalescing operator (??)
  String displayName = nullableName ?? 'Guest User';
  print('Display Name (Fallback): $displayName');

  // 3. Null-aware method/property call (?.)
  print('Name Length (Safe Call): ${nullableName?.length}');

  // Assign value
  nullableName = "Flutter Developer";

  // 4. Null assertion operator (!)
  print('Asserted Length (!): ${nullableName!.length}');

  // 5. Late initialization
  late String serverResponse;
  serverResponse = "HTTP 200 OK";
  print('Late Initialized Variable: $serverResponse');
}
```

Sample Output:

Display Name (Fallback): Guest User

Name Length (Safe Call): null

Asserted Length (!): 17

Late Initialized Variable: HTTP 200 OK

Program 8(a): Basic Student Class**Objective:**

To design a basic Object-Oriented **Student** class with attributes, a constructor, and a method to print details.

Theory & Procedure:

1. **Theory:** Encapsulation packages data fields and methods operating on that data within a single class unit.
2. **Procedure:** Define class **Student** with fields **name** and **marks**. Create a generative constructor to initialize fields and a **displayDetails()** method to print student information.

// Program 8(a): Basic Student Class

```
class Student {
    String name;
    int marks;

    // Generative Constructor
    Student(this.name, this.marks);

    void displayDetails() {
        print('Student Name : $name');
        print('Marks      : $marks');
        print('Status      : ${marks >= 40 ? "Passed" : "Failed"}\n');
    }
}

void main() {
    Student s1 = Student('Kiran', 82);
    Student s2 = Student('Suresh', 35);

    print('--- Student Directory ---');
    s1.displayDetails();
    s2.displayDetails();
}
```

Sample Output:

--- Student Directory ---

Student Name : Kiran

Marks : 82

Status : Passed

Student Name : Suresh

Marks : 35

Status : Failed

Program 8(b): Encapsulated Rectangle Class

Objective:

To build an encapsulated `Rectangle` class with private fields, constructor validation, and member methods for area and perimeter calculations.

Theory & Procedure:

1. **Theory:** Prefixing a variable or field name with an underscore (`_`) makes it library-private in Dart.
2. **Procedure:** Define `_length` and `_width` as private fields. Provide getters for properties and methods `calculateArea()` and `calculatePerimeter()`.

// Program 8(b): Rectangle Class

```
class Rectangle {  
  double _length;  
  double _width;  
  
  Rectangle(this._length, this._width);  
  
  // Getters  
  double get length => _length;  
  double get width => _width;  
  
  double calculateArea() => _length * _width;  
  double calculatePerimeter() => 2 * (_length + _width);  
}  
  
void main() {  
  Rectangle rect = Rectangle(8.0, 4.5);  
  
  print('Rectangle Dimensions : ${rect.length} x ${rect.width}');  
  print('Calculated Area : ${rect.calculateArea()} sq units');  
  print('Calculated Perimeter : ${rect.calculatePerimeter()} units');  
}
```

Sample Output:

Rectangle Dimensions : 8.0 x 4.5
Calculated Area : 36.0 sq units
Calculated Perimeter : 25.0 units

Program 9(a): Single Inheritance Architecture

Objective:

To implement single inheritance where a derived class `Student` inherits properties from a base class `Person`.

Theory & Procedure:

1. **Theory:** Inheritance enables code reusability using the `extends` keyword. The `super` constructor call initializes base class properties before derived class fields.
2. **Procedure:** Define base class `Person` with `name` and `age`. Derive class `Student` adding `studentId`. Invoke `super(name, age)` in the derived constructor.

// Program 9(a): Inheritance (Person -> Student)

```
class Person {
    String name;
    int age;

    Person(this.name, this.age);

    void displayPersonInfo() {
        print('Name : $name');
        print('Age : $age years');
    }
}

class Student extends Person {
    String studentId;

    // Invoking superclass constructor
    Student(String name, int age, this.studentId) : super(name, age);

    void displayStudentInfo() {
        displayPersonInfo(); // Inherited method call
        print('ID : $studentId\n');
    }
}

void main() {
    print('--- Student Profile (Derived Class) ---');
    Student st = Student('Anitha', 20, '21KITCS045');
    st.displayStudentInfo();
}
```

Sample Output:

```
--- Student Profile (Derived Class) ---
Name : Anitha
```

Age : 20 years
ID : 21KITCS045

Program 9(b): Division-by-Zero Exception Handling

Objective:

To demonstrate runtime exception handling using `try`, `catch`, `on`, and `finally` blocks during integer division by zero.

Theory & Procedure:

1. **Theory:** Dart uses exceptions for runtime error control. Integer division (`~/`) by zero throws a runtime exception/error that can be handled using `try-catch`.
2. **Procedure:** Attempt `10 ~/ 0` inside a `try` block, catch runtime errors using `catch (e)`, and execute cleanup logic in a `finally` block.

// Program 9(b): Division-by-Zero Exception Handling

```
void divideNumbers(int a, int b) {  
  print('Executing: $a ~/ $b');  
  try {  
    int result = a ~/ b;  
    print('Result: $result');  
  } catch (e) {  
    print('Caught Exception/Error: $e');  
  } finally {  
    print('Division execution completed.\n');  
  }  
}
```

```
void main() {  
  divideNumbers(20, 4);  
  divideNumbers(10, 0);  
}
```

Sample Output:

Executing: 20 ~/ 4

Result: 5

Division execution completed.

Executing: 10 ~/ 0

Caught Exception/Error: Unsupported operation: Result of integer division cannot be calculated.

Division execution completed.

Program 10(a): Asynchronous Futures & Timers

Objective:

To execute asynchronous operations using `Future`, `Future.delayed`, and the `async/await` keywords.

Theory & Procedure:

1. **Theory:** Asynchronous programming in Dart allows non-blocking operations using `Future` objects representing delayed computation results.
2. **Procedure:** Define an `async` function `fetchData()` that waits for a 2-second delay via `Future.delayed()`, returns a string, and is consumed using `await`.

// Program 10(a): Asynchronous Futures & async/await

```
Future<String> fetchUserData() async {  
  print('Fetching data from remote server...');  
  // Simulating network latency (2 seconds)  
  await Future.delayed(const Duration(seconds: 2));  
  return 'User Data Payload: { id: 101, status: "Active" }';  
}  
  
void main() async {  
  print('Program started.');  String data = await fetchUserData();  
  print('Received Response: $data');  print('Program finished.');}
```

Sample Output:

```
Program started.  
Fetching data from remote server...  
(2 second delay...)  
Received Response: User Data Payload: { id: 101, status: "Active" }  
Program finished.
```

Program 10(b): Higher-Order & Anonymous Functions

Objective:

To demonstrate higher-order functions, anonymous functions (lambdas), and list iteration using `.forEach()`.

Theory & Procedure:

1. **Theory:** Functions in Dart are first-class objects. A higher-order function takes another function as a parameter or returns a function. Anonymous functions are unnamed function literals.
2. **Procedure:** Define a higher-order function `processNumbers()`. Pass an anonymous function to transform a list of numbers and use `.forEach()` to print the results.

// Program 10(b): Higher-Order & Anonymous Functions

```
// Higher-order function accepting a transformation callback  
void processList(List<int> numbers, int Function(int) action) {
```

```

    for (int n in numbers) {
        int transformed = action(n);
        print('Original: $n -> Transformed: $transformed');
    }
}

void main() {
    List<int> dataset = [1, 2, 3, 4, 5];

    print('--- Higher-Order Function Execution (Squaring) ---');
    // Passing anonymous lambda (x => x * x)
    processList(dataset, (x) => x * x);

    print('\n--- Collection Iteration with .forEach() ---');
    // Anonymous function inside forEach
    dataset.forEach((element) {
        print('Item: $element');
    });
}

```

Sample Output:

--- Higher-Order Function Execution (Squaring) ---

Original: 1 -> Transformed: 1
 Original: 2 -> Transformed: 4
 Original: 3 -> Transformed: 9
 Original: 4 -> Transformed: 16
 Original: 5 -> Transformed: 25

--- Collection Iteration with .forEach() ---

Item: 1
 Item: 2
 Item: 3
 Item: 4
 Item: 5

Part II: Restructured Flutter Experiments —

Experiment 1(a): Install Flutter and Dart SDK Environment Verification

Objective:

To install the Flutter and Dart SDKs, configure system environment PATH variables, run diagnostic checks ([flutter doctor](#)), and execute a baseline application to verify compilation.

Theory & Procedure:

1. Installation & PATH Configuration:

- **Download & Extract:** Download the latest stable Flutter SDK package from flutter.dev and extract it to a local directory (e.g., `C:\src\flutter` on Windows or `~/development/flutter` on macOS/Linux).
- **Set PATH Environment Variable:** Add the absolute path of the `flutter/bin` directory to your OS Environment PATH variables so the `flutter` and `dart` executables are globally accessible.
- **Restart Terminal:** Reopen the command prompt or terminal session to reload PATH settings.

2. Diagnostic Verification & Execution:

- Execute `flutter doctor` in the terminal to verify that the Flutter framework, standalone Dart SDK, Android/iOS toolchains, and target emulators/browsers are operational.
- Compile and launch a minimal Flutter widget tree in `main.dart` to confirm engine rendering.

Terminal Diagnostic Commands

`flutter --version`

`dart --version`

`flutter doctor`

// Experiment 1(a): SDK Verification App

```
import 'package:flutter/material.dart';
```

```
void main() {
```

```
  runApp(const SdkVerificationApp());
```

```
}
```

```
class SdkVerificationApp extends StatelessWidget {
```

```
  const SdkVerificationApp({super.key});
```

```
  @override
```

```
  Widget build(BuildContext context) {
```

```
    return MaterialApp(
```

```
      debugShowCheckedModeBanner: false,
```

```
      title: 'Exp 1(a): SDK Verification',
```

```
      home: Scaffold(
```

```
        appBar: AppBar(
```

```
          title: const Text('SDK Environment Verification'),
```

```
          backgroundColor: Colors.blueAccent,
```

```
        ),
```

```
        body: const Center(
```

```
          child: Column(
```

```
            mainAxisAlignment: MainAxisAlignment.center,
```

```
            children: [
```

```
              Icon(Icons.check_circle_outline, color: Colors.green, size: 72),
```

```
              SizedBox(height: 16),
```

```
              Text(
```

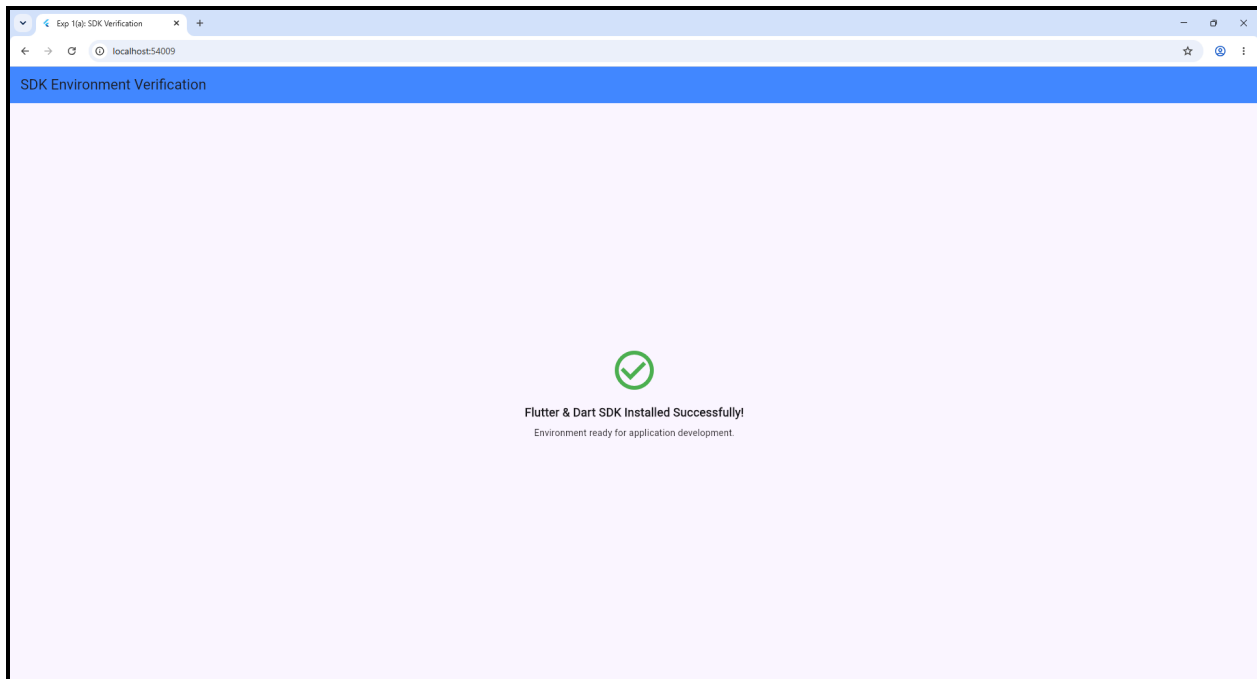
```
                'Flutter & Dart SDK Installed Successfully!',
```

```
                style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
```

```

    ),
    SizedBox(height: 8),
    Text('Environment ready for application development.'),
  ],
),
),
),
),
);
}
}

```



Experiment 1(b): Simple Dart Program & Language Basics

Objective:

To write and execute a simple standalone Dart console program demonstrating primitive data types, arithmetic calculations, conditional grade evaluation, and formatted output.

Theory & Procedure:

1. **Theory:** Dart is an object-oriented language supporting static typing and type inference. Standalone Dart programs execute from a pure `main()` entry point independently of the Flutter rendering engine framework.
2. **Procedure:**
 - Declare typed variables (`String`, `int`, `double`).
 - Perform arithmetic calculations to compute total and average marks.
 - Evaluate conditional expressions using a Dart `switch` expression to determine grades.

- Print formatted results to standard output using `print()`.

// Experiment 1(b): Simple Dart Program & Language Basics

```
void main() {  
  String studentName = "Aarav Sharma";  
  int mark1 = 85;  
  int mark2 = 90;  
  int mark3 = 78;  
  
  int totalMarks = mark1 + mark2 + mark3;  
  double averageMarks = totalMarks / 3.0;  
  
  String grade = switch (averageMarks) {  
    >= 85 => 'Distinction',  
    >= 70 => 'First Class',  
    >= 50 => 'Second Class',  
    _ => 'Pass',  
  };  
  
  print('--- Student Performance Report ---');  
  print('Student Name : $studentName');  
  print('Marks Scored : $mark1, $mark2, $mark3');  
  print('Total Marks : $totalMarks / 300');  
  print('Average Marks : ${averageMarks.toStringAsFixed(2)}%');  
  print('Final Grade : $grade');  
}
```

Sample Output:

```
--- Student Performance Report ---  
Student Name : Aarav Sharma  
Marks Scored : 85, 90, 78  
Total Marks : 253 / 300  
Average Marks : 84.33%  
Final Grade : First Class
```

Experiment 2(a): Explore Basic Widgets (Text, Image, Container)

Objective:

To construct a visual user interface incorporating foundational widgets: `Text`, `Image.network`, and `Container` with box decorations.

Theory & Procedure:

1. **Theory:** `Text` displays styled character strings. `Container` provides rectangular box model capabilities including padding, margin, border radius, and colors. `Image.network` downloads and renders external images asynchronously over HTTP.
2. **Procedure:**
 - Nest a `Container` with explicit padding and border radius inside a `Center` widget.

- Add a `Text` title with customized typography.
- Render a remote asset using `Image.network()`.

// Experiment 2(a): Text, Image, and Container Widgets

```
import 'package:flutter/material.dart';

void main() {
  runApp(const BasicWidgetsApp());
}

class BasicWidgetsApp extends StatelessWidget {
  const BasicWidgetsApp({super.key});

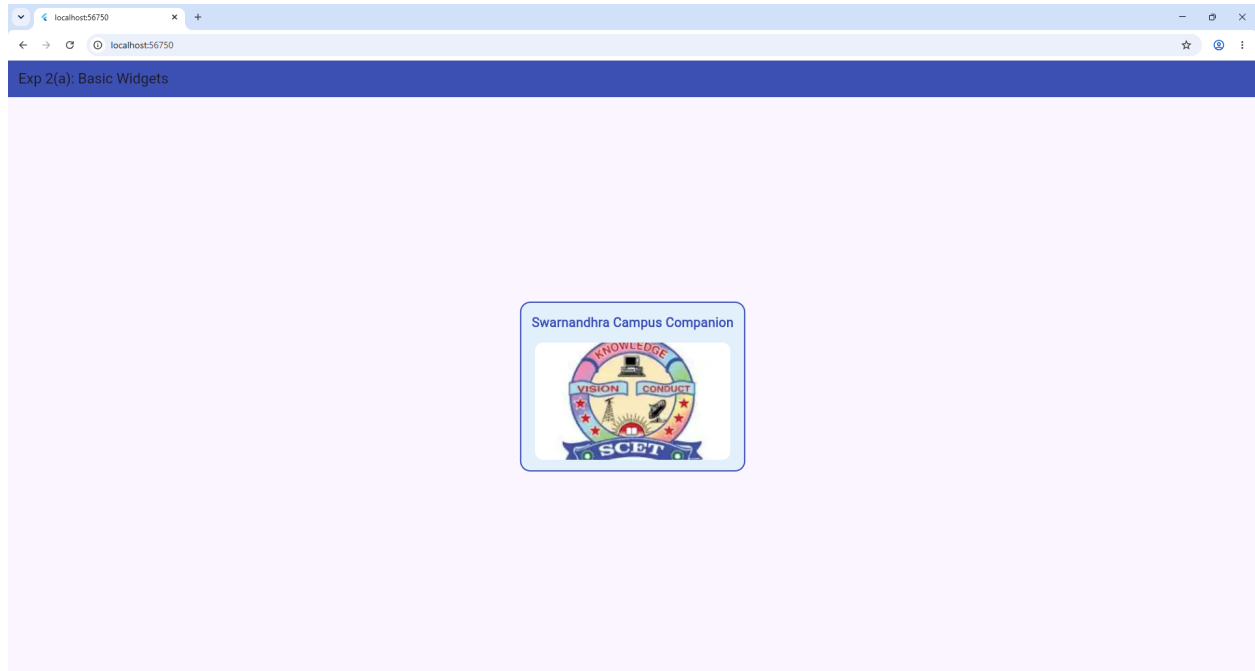
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Exp 2(a): Basic Widgets'),
          backgroundColor: Colors.indigo,
        ),
        body: Center(
          child: Container(
            padding: const EdgeInsets.all(16.0),
            margin: const EdgeInsets.all(16.0),
            decoration: BoxDecoration(
              color: Colors.blue.shade50,
              borderRadius: BorderRadius.circular(16.0),
              border: Border.all(color: Colors.indigo, width: 2.0),
            ),
            child: Column(
              mainAxisAlignment: MainAxisAlignment.min,
              children: [
                const Text(
                  'Swarnandhra Campus Companion',
                  style: TextStyle(
                    fontSize: 20,
                    fontWeight: FontWeight.bold,
                    color: Colors.indigo,
                  ),
                ),
              ],
            ),
          ),
        ),
      ),
    );
  }
}
```

```

    ),
    const SizedBox(height: 16),
    ClipRRect(
      borderRadius: BorderRadius.circular(12),
      child: Image.network(
        'https://tse1.mm.bing.net/th/id/OIP.6d2DkXmKofRhXeUANh4XPAAAAA?r=0&rs=1&pid=ImgDetMain&o=7&rm=3',
        width: 300,
        height: 180,
        fit: BoxFit.cover,
      ),
    ),
  ],
),
),
);
}
}

```

Note: Requires active internet connectivity to fetch the remote asset.



Experiment 2(b): Multi-Child Layout Mechanics (Row, Column, Stack)

Objective:

To construct complex UI layouts by combining 1D axis widgets (**Row**, **Column**) and Z-axis layering (**Stack**).

Theory & Procedure:

1. **Theory:** **Row** arranges elements horizontally and **Column** arranges elements vertically along 1D axes. **Stack** overlays children on top of each other along the Z-axis using absolute or alignment positioning.
2. **Procedure:**
 - Arrange icons horizontally inside a **Row**.
 - Layer overlapping rectangular containers and labels inside a **Stack**.
 - Organize all layout sections vertically inside a parent **Column**.

// Experiment 2(b): Row, Column, and Stack Layouts

```
import 'package:flutter/material.dart';
```

```
void main() {  
  runApp(const LayoutStructuresApp());  
}
```

```
class LayoutStructuresApp extends StatelessWidget {  
  const LayoutStructuresApp({super.key});
```

```
  @override
```

```

Widget build(BuildContext context) {
  return MaterialApp(
    debugShowCheckedModeBanner: false,
    home: Scaffold(
      appBar: AppBar(title: const Text('Exp 2(b): Layout Structures')),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Column(
          children: [
            // 1. Row Layout Demonstration
            const Text('Row Layout (Horizontal Axis)', style: TextStyle(fontWeight: FontWeight.bold)),
            const SizedBox(height: 8),
            Row(
              mainAxisAlignment: MainAxisAlignment.spaceEvenly,
              children: const [
                Icon(Icons.star, color: Colors.amber, size: 36),
                Icon(Icons.star, color: Colors.amber, size: 36),
                Icon(Icons.star, color: Colors.amber, size: 36),
              ],
            ),
            const Divider(height: 32, thickness: 2),
            // 2. Stack Layout Demonstration
            const Text('Stack Layout (Z-Axis Overlapping)', style: TextStyle(fontWeight:
FontWeight.bold)),
            const SizedBox(height: 12),
            Stack(
              alignment: Alignment.center,
              children: [
                Container(width: 180, height: 180, color: Colors.blue.shade200),
                Container(width: 130, height: 130, color: Colors.blue.shade400),
                Container(
                  padding: const EdgeInsets.all(8),
                  color: Colors.indigo,
                  child: const Text('Stacked Label', style: TextStyle(color: Colors.white)),
                ),
              ],
            ),
          ],
        ),
      ),
    ),
  );
}

```



Experiment 3(a): Responsive UI Architecture (LayoutBuilder)

Objective:

To implement adaptive, responsive UI layouts that restructure dynamically based on parent container constraints using `LayoutBuilder`.

Theory & Procedure:

1. **Theory:** `LayoutBuilder` evaluates `BoxConstraints` passed by parent widgets. Comparing `constraints.maxWidth` against threshold breakpoints allows switching between single-column and multi-column view layouts.
2. **Procedure:**
 - Wrap layout body inside a `LayoutBuilder`.
 - Evaluate `constraints.maxWidth` against a 600px breakpoint.
 - Return a two-column `Row` layout for wide viewports and a single-column `Column` layout for narrow viewports.

// Experiment 3(a): Responsive UI using LayoutBuilder

```
import 'package:flutter/material.dart';
```

```
void main() {
  runApp(const ResponsiveUiApp());
}
```

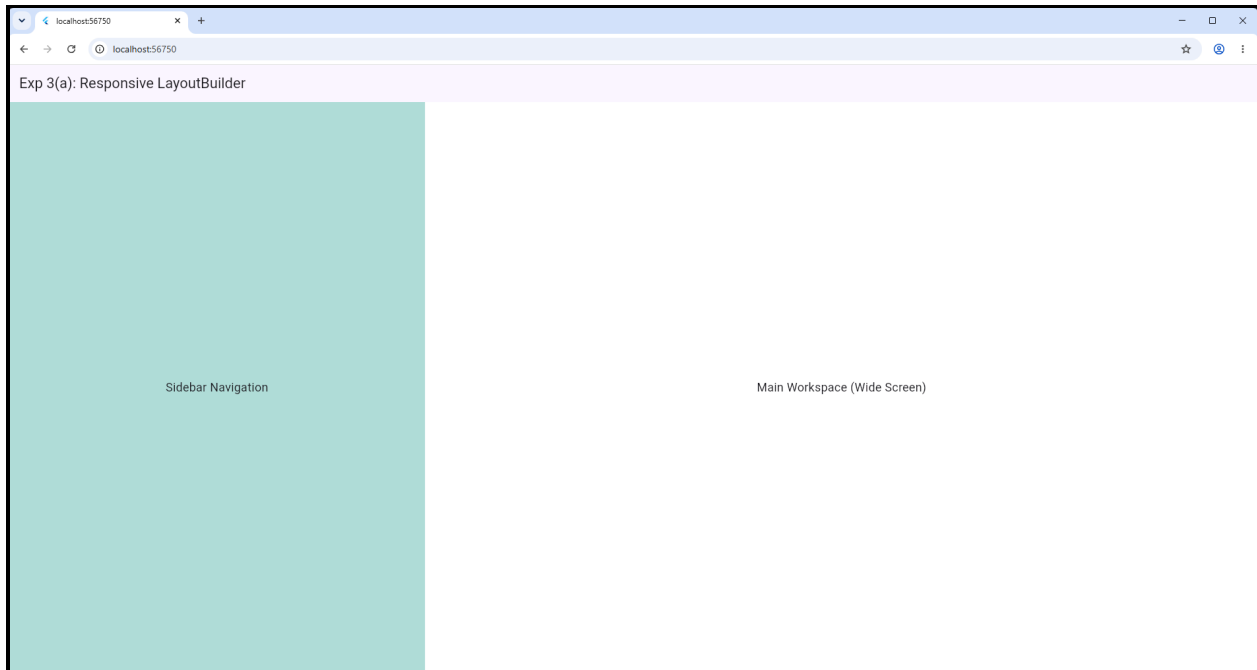
```
class ResponsiveUiApp extends StatelessWidget {
  const ResponsiveUiApp({super.key});
```

```
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(title: const Text('Exp 3(a): Responsive LayoutBuilder')),
        body: LayoutBuilder(
          builder: (context, constraints) {
            if (constraints.maxWidth >= 600) {
              // Wide Viewport Layout (Tablet / Desktop)
```

```

        return Row(
          children: [
            Expanded(
              child: Container(
                color: Colors.teal.shade100,
                child: const Center(child: Text('Sidebar Navigation', style: TextStyle(fontSize: 18))),
              ),
            ),
            Expanded(
              flex: 2,
              child: Container(
                color: Colors.white,
                child: const Center(child: Text('Main Workspace (Wide Screen)', style:
TextStyle(fontSize: 18))),
              ),
            ),
          ],
        );
      } else {
        // Narrow Viewport Layout (Mobile)
        return Column(
          children: [
            Container(
              height: 80,
              color: Colors.teal.shade100,
              child: const Center(child: Text('Header Banner')),
            ),
            Expanded(
              child: Container(
                color: Colors.white,
                child: const Center(child: Text('Main Content Area (Mobile View)', style:
TextStyle(fontSize: 18))),
              ),
            ),
          ],
        );
      }
    },
  ),
),
);
}
},
),
),
);
}
}
}

```



Experiment 3(b): Media Queries & Screen Breakpoints

Objective:

To query device display metrics (screen dimensions, aspect ratio, orientation) using `MediaQuery.of(context)` and adjust visual styling dynamically.

Theory & Procedure:

1. **Theory:** `MediaQuery.of(context)` provides access to media parameters such as `size.width`, `size.height`, and `orientation`.
2. **Procedure:**
 - Obtain display dimensions via `MediaQuery.of(context).size`.
 - Set breakpoint flags (e.g., `isTablet = width >= 600`).
 - Adapt font sizes, padding, and layout orientation dynamically based on metrics.

// Experiment 3(b): Media Queries & Breakpoints

```
import 'package:flutter/material.dart';
```

```
void main() {  
  runApp(const MediaQueryApp());  
}
```

```
class MediaQueryApp extends StatelessWidget {  
  const MediaQueryApp({super.key});
```

```
  @override  
  Widget build(BuildContext context) {  
    return const MaterialApp(  

```

```

        debugShowCheckedModeBanner: false,
        home: MediaQueryBreakpointScreen(),
    );
}
}

```

```

class MediaQueryBreakpointScreen extends StatelessWidget {
  const MediaQueryBreakpointScreen({super.key});

```

```

  @override
  Widget build(BuildContext context) {
    Size screenSize = MediaQuery.of(context).size;
    Orientation orientation = MediaQuery.of(context).orientation;
    bool isTablet = screenSize.width >= 600;

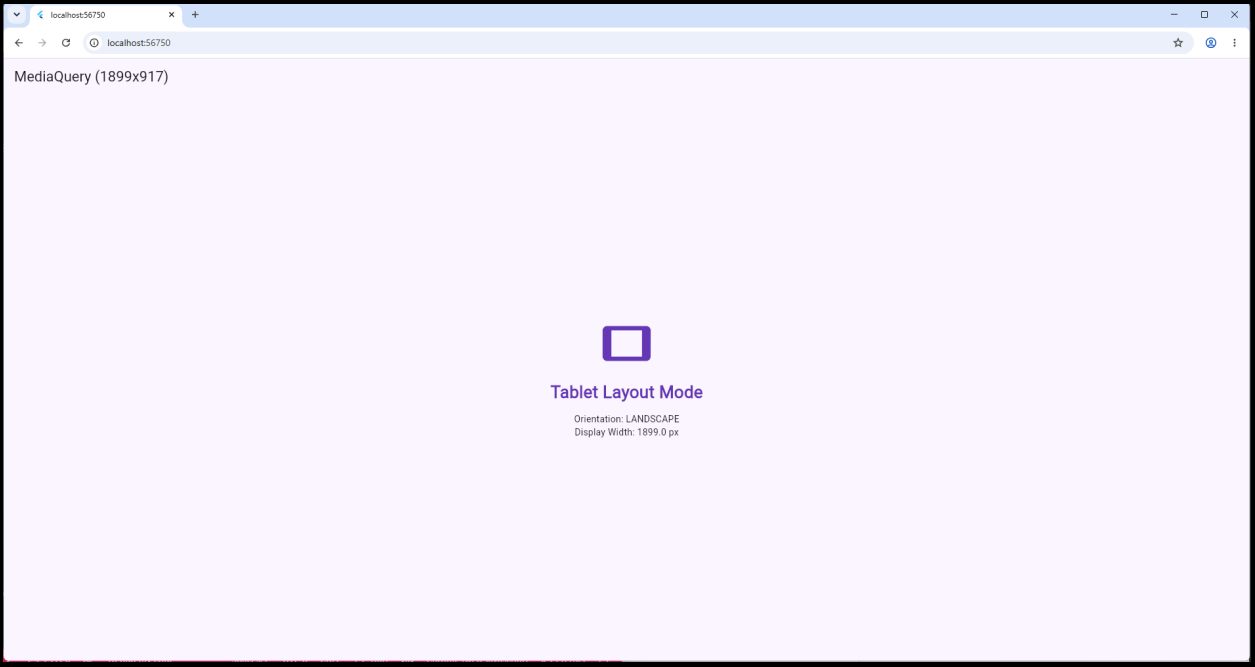
```

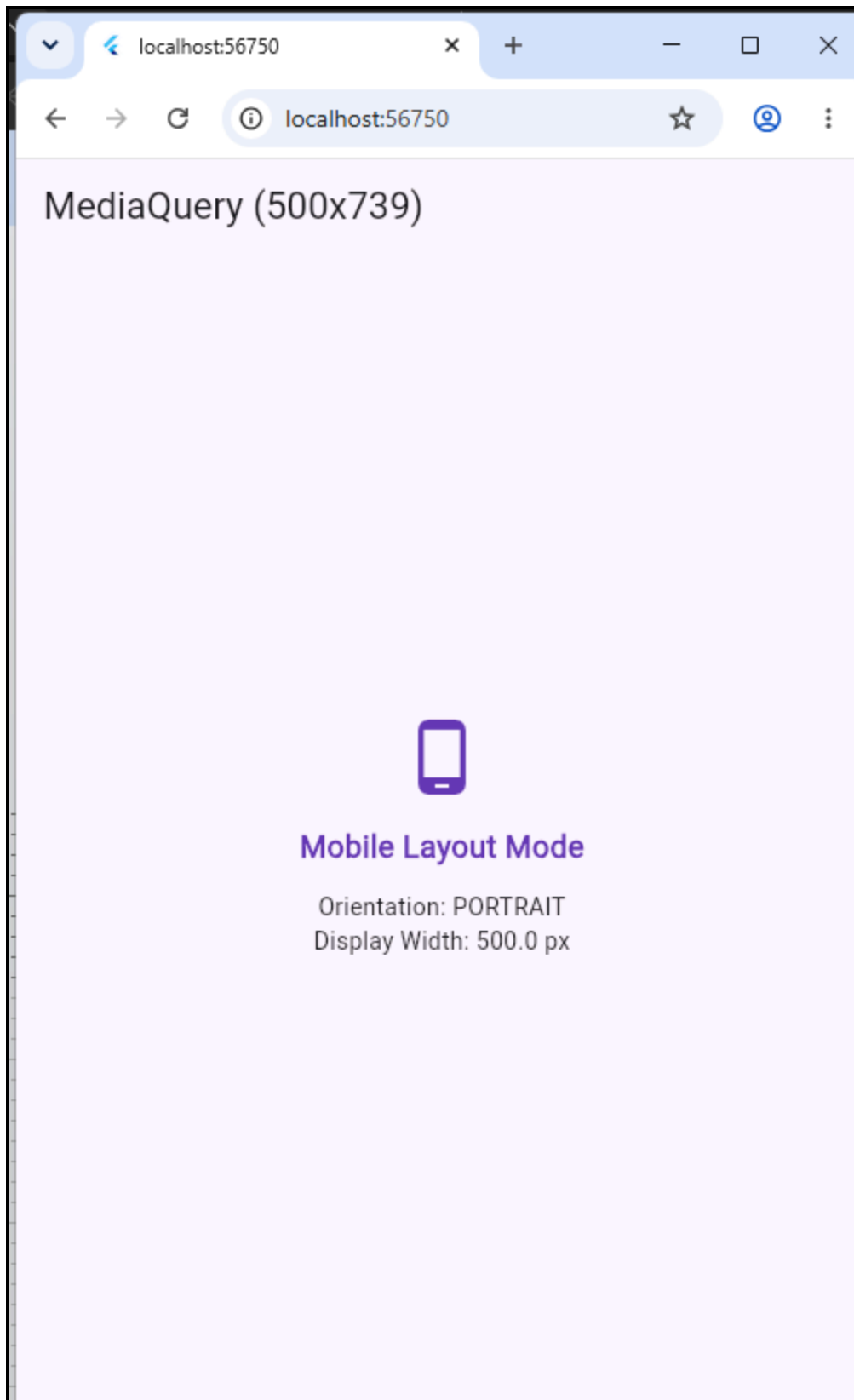
```

    return Scaffold(
      appBar: AppBar(
        title: Text('MediaQuery (${screenSize.width.toInt()}x${screenSize.height.toInt()})'),
      ),
      body: Center(
        child: Padding(
          padding: const EdgeInsets.all(20.0),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              Icon(
                isTablet ? Icons.tablet : Icons.phone_android,
                size: isTablet ? 80 : 48,
                color: Colors.deepPurple,
              ),
              const SizedBox(height: 16),
              Text(
                isTablet ? 'Tablet Layout Mode' : 'Mobile Layout Mode',
                style: TextStyle(
                  fontSize: isTablet ? 26 : 18,
                  fontWeight: FontWeight.bold,
                  color: Colors.deepPurple,
                ),
              ),
              const SizedBox(height: 12),
              Text('Orientation: ${orientation.name.toUpperCase()}'),
              Text('Display Width: ${screenSize.width.toStringAsFixed(1)} px'),
            ],
          ),
        ),
      ),
    ),
  ),
)

```

);
}
}





Experiment 4(a): Imperative Navigator Stack Operations

Objective:

To transition between application screens using imperative stack push and pop mechanics (`Navigator.push`, `Navigator.pop`).

Theory & Procedure:

1. **Theory:** `Navigator` manages a LIFO route stack. `Navigator.push()` places a new `MaterialPageRoute` on top of the stack, and `Navigator.pop()` removes the top route to return to the preceding screen.
2. **Procedure:**
 - Define `FirstScreen` and `SecondScreen` widgets.
 - Trigger `Navigator.push(context, MaterialPageRoute(...))` on button tap.
 - Trigger `Navigator.pop(context)` on the target page to return.

// Experiment 4(a): Imperative Navigator Routing

```
import 'package:flutter/material.dart';
```

```
void main() {  
  runApp(const ImperativeNavApp());  
}
```

```
class ImperativeNavApp extends StatelessWidget {  
  const ImperativeNavApp({super.key});
```

```
  @override  
  Widget build(BuildContext context) {  
    return const MaterialApp(  
      debugShowCheckedModeBanner: false,  
      home: FirstScreen(),  
    );  
  }  
}
```

```
class FirstScreen extends StatelessWidget {  
  const FirstScreen({super.key});
```

```
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: const Text("First Screen (Home)")),  
      body: Center(  
        child: ElevatedButton(  
          onPressed: () {  
            Navigator.push(  
              context,  
              MaterialPageRoute(builder: (context) => const SecondScreen()),  
            );  
          },  
        ),  
      ),  
    );  
  }  
}
```

```

    );
  },
  child: const Text('Navigate to Second Screen'),
),
),
);
}
}

```

```

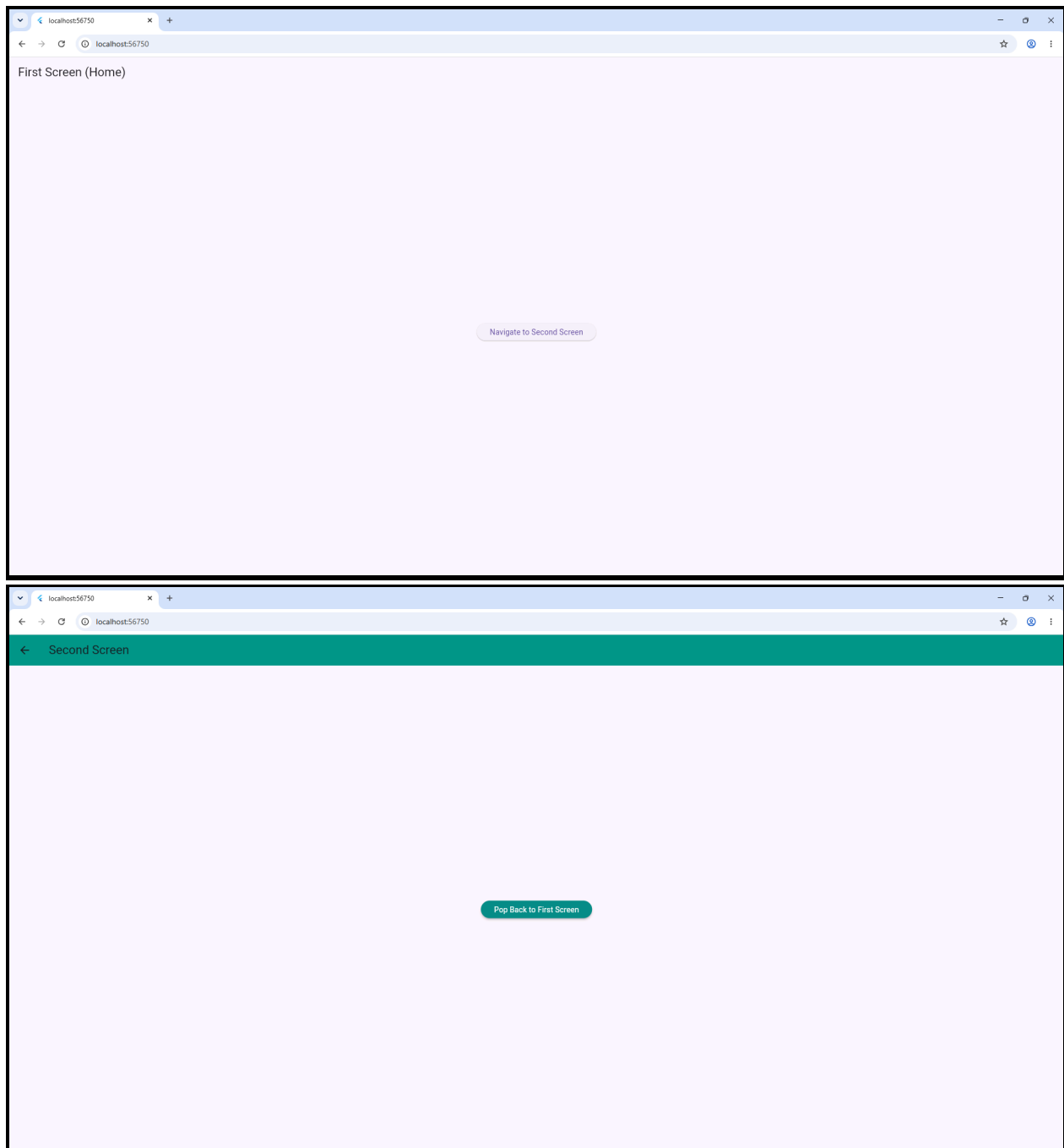
class SecondScreen extends StatelessWidget {
  const SecondScreen({super.key});

```

```

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Second Screen'), backgroundColor: Colors.teal),
      body: Center(
        child: ElevatedButton(
          style: ElevatedButton.styleFrom(backgroundColor: Colors.teal),
          onPressed: () => Navigator.pop(context),
          child: const Text('Pop Back to First Screen', style: TextStyle(color: Colors.white)),
        ),
      ),
    );
  }
}

```



Experiment 4(b): Named Routes Navigation Architecture

Objective:

To configure centralized application navigation using `initialRoute`, the `routes` table, and `Navigator.pushNamed`.

Theory & Procedure:

1. **Theory:** Named routes centralize path mappings within the `MaterialApp` configuration, decoupling screen implementations from explicit class references across navigation targets.
2. **Procedure:**
 - Define string route keys (e.g., `'/'`, `'/details'`) inside `MaterialApp.routes`.
 - Set `initialRoute: '/'`.
 - Invoke `Navigator.pushNamed(context, '/details')` to initiate transitions.

// Experiment 4(b): Named Routes Navigation Architecture

```
import 'package:flutter/material.dart';
```

```
void main() {  
  runApp(MaterialApp(  
    debugShowCheckedModeBanner: false,  
    initialRoute: '/',  
    routes: {  
      '/': (context) => const NamedHomeScreen(),  
      '/details': (context) => const NamedDetailsScreen(),  
    },  
  ));  
}
```

```
class NamedHomeScreen extends StatelessWidget {  
  const NamedHomeScreen({super.key});
```

```
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: const Text('Named Route Home')),  
      body: Center(  
        child: ElevatedButton(  
          onPressed: () => Navigator.pushNamed(context, '/details'),  
          child: const Text('Open Details Route (/details)'),  
        ),  
      ),  
    );  
  }  
}
```

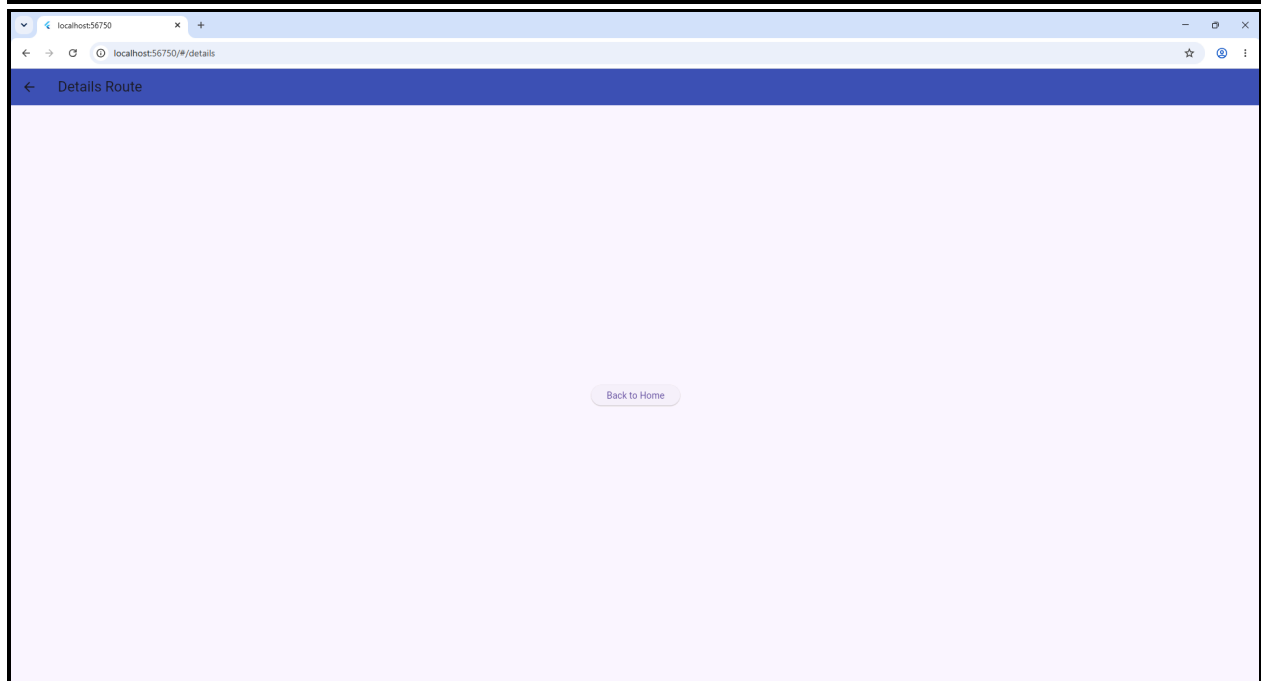
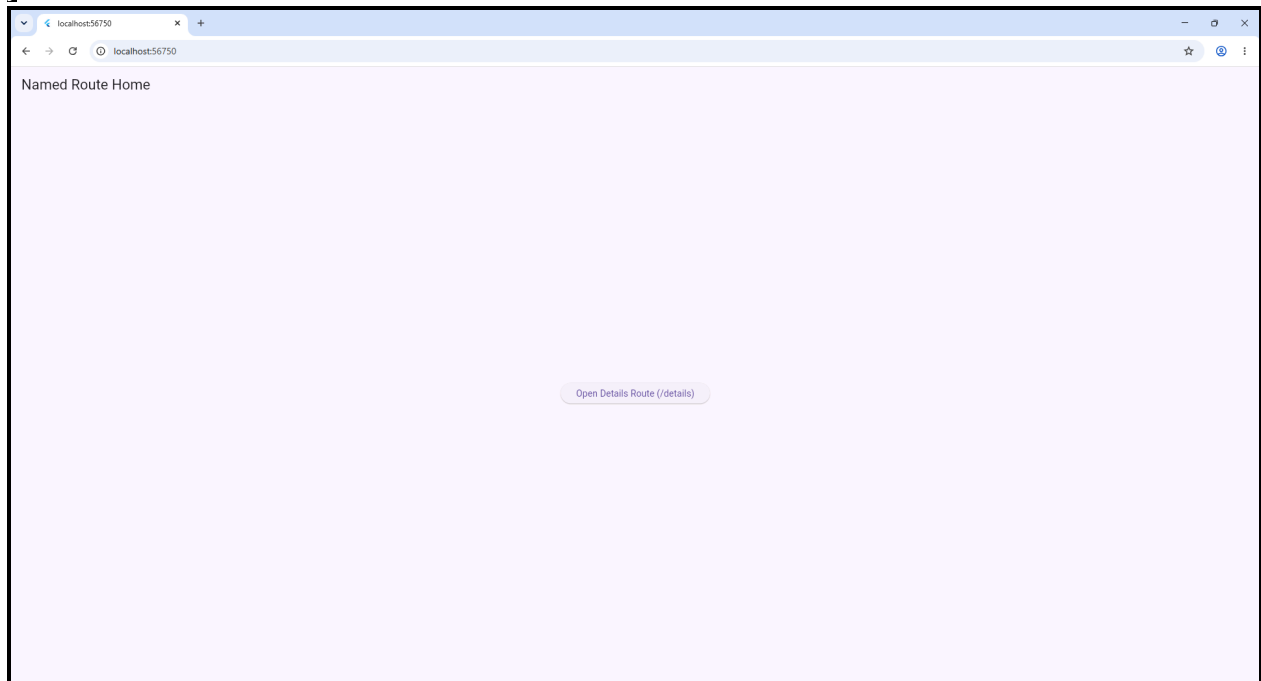
```
class NamedDetailsScreen extends StatelessWidget {  
  const NamedDetailsScreen({super.key});
```

```
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: const Text('Details Route'), backgroundColor: Colors.indigo),  
      body: Center(  
        child: ElevatedButton(  
          onPressed: () => Navigator.pushNamed(context, '/'),  
          child: const Text('Open Home Route (/)'),  
        ),  
      ),  
    );  
  }  
}
```

```

onPressed: () => Navigator.pop(context),
child: const Text('Back to Home'),
),
),
);
}
}

```



Experiment 5(a): Stateless vs. Stateful Widgets Lifecycle

Objective:

To compare immutable `StatelessWidget` and dynamic `StatefulWidget` mechanics in a single application.

Theory & Procedure:

1. **Theory:** `StatelessWidget` renders fixed UI based solely on constructor arguments. `StatefulWidget` delegates rendering to a persistent `State` object whose fields mutate inside `setState()` calls.
2. **Procedure:**
 - Create a `StatelessWidget` displaying static header parameters.
 - Create a `StatefulWidget` managing a counter state variable updated via `setState()`.
 - Combine both components inside a single view layout.

// Experiment 5(a): Stateless vs Stateful Widgets

```
import 'package:flutter/material.dart';
```

```
void main() {  
  runApp(const WidgetStateDemoApp());  
}
```

```
class WidgetStateDemoApp extends StatelessWidget {  
  const WidgetStateDemoApp({super.key});
```

```
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      debugShowCheckedModeBanner: false,  
      home: Scaffold(  
        appBar: AppBar(title: const Text('Exp 5(a): Widget State Dynamics')),  
        body: const Center(  
          child: Column(  
            mainAxisAlignment: MainAxisAlignment.center,  
            children: [  
              StaticHeader(title: 'Stateless vs Stateful Demo'),  
              SizedBox(height: 24),  
              DynamicCounterWidget(),  
            ],  
          ),  
        ),  
      ),  
    );  
  }  
}
```

// 1. Immutable Stateless Component

```
class StaticHeader extends StatelessWidget {
```

```

final String title;
const StaticHeader({super.key, required this.title});

@override
Widget build(BuildContext context) {
  return Text(
    title,
    style: const TextStyle(fontSize: 20, fontWeight: FontWeight.bold),
  );
}
}

```

// 2. Mutable Stateful Component

```

class DynamicCounterWidget extends StatefulWidget {
  const DynamicCounterWidget({super.key});
}

```

```

@override
State<DynamicCounterWidget> createState() => _DynamicCounterWidgetState();
}

```

```

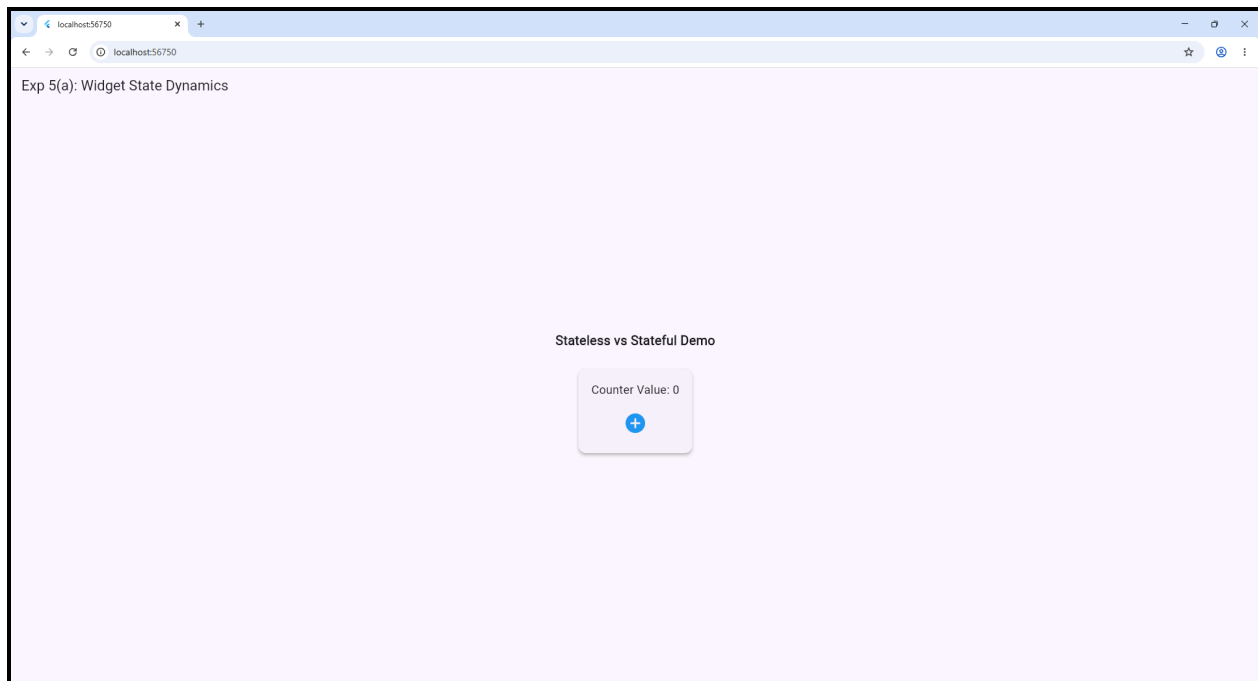
class _DynamicCounterWidgetState extends State<DynamicCounterWidget> {
  int _counter = 0;
}

```

```

@override
Widget build(BuildContext context) {
  return Card(
    elevation: 4,
    child: Padding(
      padding: const EdgeInsets.all(20.0),
      child: Column(
        children: [
          Text('Counter Value: $_counter', style: const TextStyle(fontSize: 18)),
          const SizedBox(height: 12),
          IconButton(
            icon: const Icon(Icons.add_circle, size: 36, color: Colors.blue),
            onPressed: () => setState(() => _counter++),
          ),
        ],
      ),
    ),
  );
}
}

```



Experiment 5(b): Dual State Management (setState and Provider)

Objective:

To demonstrate and contrast local component state (`setState`) and global app-wide state (`Provider`) on a single running screen.

Theory & Procedure:

1. **Theory:** `setState()` manages local, ephemeral state isolated to a single widget. `Provider` utilizes `ChangeNotifier` to propagate global state changes across the widget tree reactively via `Consumer`.
2. **Procedure:** Add `Provider` package by executing following command - `flutter pub add provider`
 - Define an app-level state class `CounterModel` extending `ChangeNotifier`.
 - Create a local counter component using `setState()`.
 - Create a global counter component using `Provider.of<CounterModel>` and `Consumer`.
 - Wrap the application root with `ChangeNotifierProvider`.

// Experiment 5(b): Dual State Management (setState & Provider)

```
import 'package:flutter/material.dart';  
import 'package:provider/provider.dart';
```

```
// --- Part 1: Shared Global State Model ---  
class CounterModel extends ChangeNotifier {  
  int _count = 0;  
  int get count => _count;
```

```

void increment() {
  _count++;
  notifyListeners();
}
}

```

```

void main() {
  runApp(
    ChangeNotifierProvider(
      create: ( ) => CounterModel(),
      child: const DualStateApp(),
    ),
  );
}

```

```

class DualStateApp extends StatelessWidget {
  const DualStateApp({super.key});

```

```

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(title: const Text('Exp 5(b): setState vs Provider')),
        body: const Padding(
          padding: EdgeInsets.all(16.0),
          child: Column(
            children: [
              LocalCounterSection(),
              Divider(height: 40, thickness: 2),
              ProviderCounterSection(),
            ],
          ),
        ),
      );
  }
}

```

```

// --- Part 2: Local Ephemeral State (setState) ---
class LocalCounterSection extends StatefulWidget {
  const LocalCounterSection({super.key});

```

```

  @override
  State<LocalCounterSection> createState() => _LocalCounterSectionState();
}

```

```

class _LocalCounterSectionState extends State<LocalCounterSection> {
  int _localCount = 0;

  @override
  Widget build(BuildContext context) {
    return Card(
      color: Colors.blue.shade50,
      child: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Column(
          children: [
            const Text('Local State (setState)', style: TextStyle(fontSize: 16, fontWeight:
FontWeight.bold)),
            const SizedBox(height: 8),
            Text('Local Count: $_localCount', style: const TextStyle(fontSize: 18)),
            const SizedBox(height: 8),
            ElevatedButton(
              onPressed: () => setState(() => _localCount++),
              child: const Text('Increment Local Counter'),
            ),
          ],
        ),
      ),
    );
  }
}

```

// --- Part 3: Global Shared State (Provider) ---

```

class ProviderCounterSection extends StatelessWidget {
  const ProviderCounterSection({super.key});

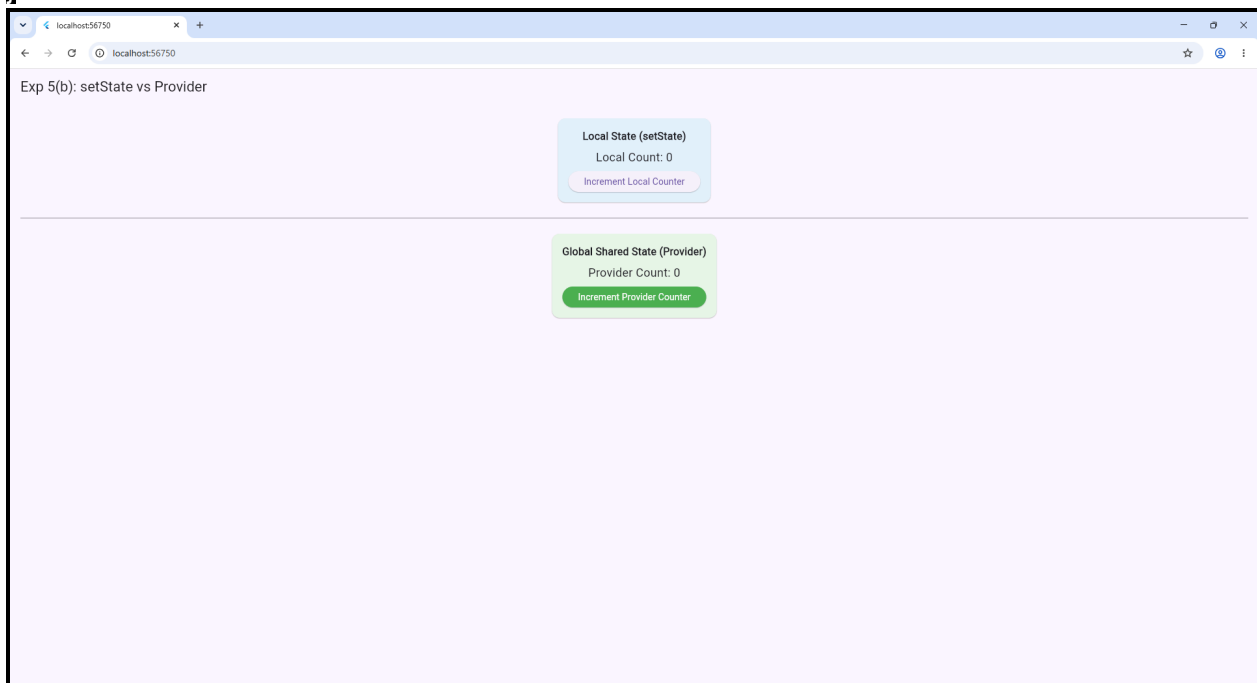
  @override
  Widget build(BuildContext context) {
    return Card(
      color: Colors.green.shade50,
      child: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Column(
          children: [
            const Text('Global Shared State (Provider)', style: TextStyle(fontSize: 16, fontWeight:
FontWeight.bold)),
            const SizedBox(height: 8),
            Consumer<CounterModel>(
              builder: (context, counter, child) {
                return Text('Provider Count: ${counter.count}', style: const TextStyle(fontSize: 18));
              },
            ),
          ],
        ),
      ),
    );
  }
}

```

```

    },
  ),
  const SizedBox(height: 8),
  ElevatedButton(
    onPressed: () => context.read<CounterModel>().increment(),
    style: ElevatedButton.styleFrom(backgroundColor: Colors.green),
    child: const Text('Increment Provider Counter', style: TextStyle(color: Colors.white)),
  ),
],
),
),
),
),
),
}
}
}

```



Experiment 6(a): Custom & Composable Widgets

Objective:

To extract and design modular, reusable custom UI components with customizable properties.

Theory & Procedure:

1. **Theory:** Reusable custom widgets encapsulate UI layout and styling logic into independent `StatelessWidget` or `StatefulWidget` classes. This prevents code duplication across screens and ensures uniform design consistency.
2. **Procedure:**

- Create an extracted widget class (e.g., `CustomInfoCard`) extending `StatelessWidget`.
- Pass custom properties (`title`, `subtitle`, `icon`) via constructor parameters.
- Instantiate multiple instances of `CustomInfoCard` inside a main screen view.

// Experiment 6(a): Custom & Composable Widgets

```
import 'package:flutter/material.dart';
```

```
void main() => runApp(const CustomWidgetApp());
```

```
class CustomWidgetApp extends StatelessWidget {
  const CustomWidgetApp({super.key});
```

```
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        appBar: AppBar(title: const Text('Custom Composable Widgets')),
        body: const Padding(
          padding: EdgeInsets.all(16.0),
          child: Column(
            children: [
              CustomInfoCard(
                title: 'Mobile Development',
                subtitle: 'Flutter & Dart SDK',
                icon: Icons.phone_android,
                badgeColor: Colors.indigo,
              ),
              SizedBox(height: 12),
              CustomInfoCard(
                title: 'Cloud Computing',
                subtitle: 'AWS & Microservices',
                icon: Icons.cloud,
                badgeColor: Colors.teal,
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

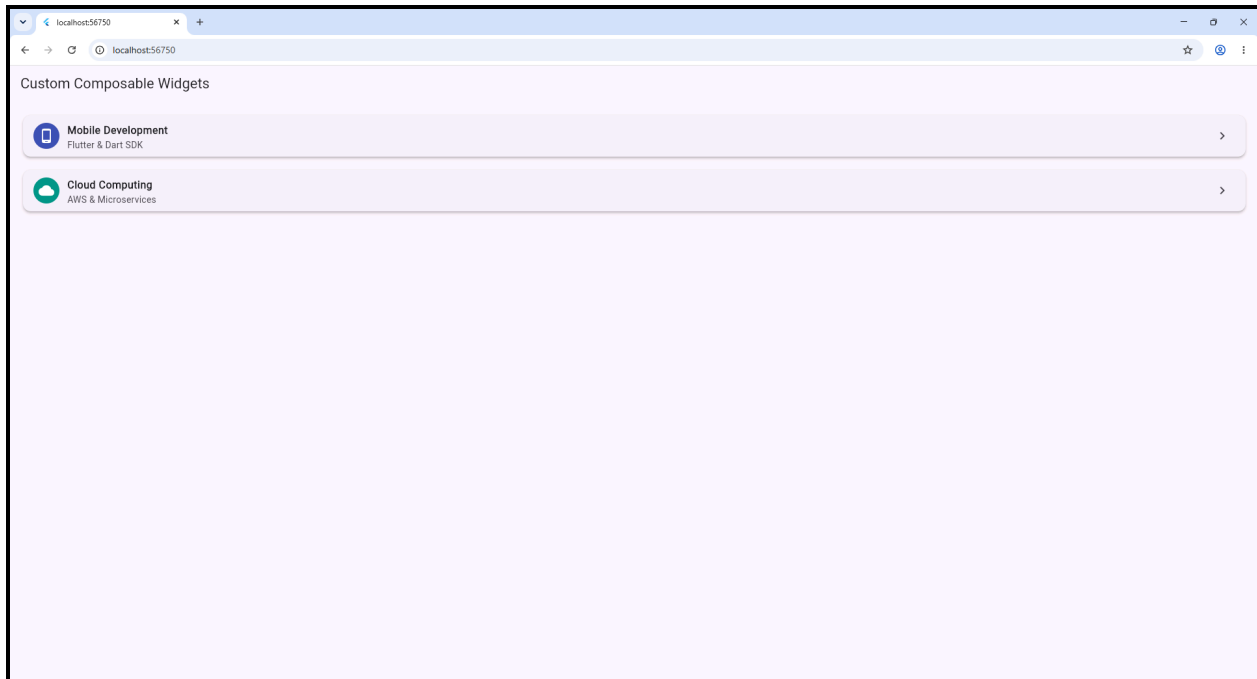
// Extracted Reusable Component

```
class CustomInfoCard extends StatelessWidget {
  final String title;
```

```
final String subtitle;  
final IconData icon;  
final Color badgeColor;
```

```
const CustomInfoCard({  
  super.key,  
  required this.title,  
  required this.subtitle,  
  required this.icon,  
  required this.badgeColor,  
});
```

```
@override  
Widget build(BuildContext context) {  
  return Card(  
    elevation: 3,  
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),  
    child: ListTile(  
      leading: CircleAvatar(  
        backgroundColor: badgeColor,  
        child: Icon(icon, color: Colors.white),  
      ),  
      title: Text(title, style: const TextStyle(fontWeight: FontWeight.bold)),  
      subtitle: Text(subtitle),  
      trailing: const Icon(Icons.chevron_right),  
    ),  
  );  
}
```



Experiment 6(b): App Themes & Dynamic Styling

Objective:

To configure global light and dark themes using `ThemeData` and toggle between themes dynamically at runtime.

Theory & Procedure:

1. **Theory:** `ThemeData` defines global visual styling across an application (colors, typography, component themes). `ThemeMode` controls whether light mode, dark mode, or system default mode is active.
2. **Procedure:**
 - Define distinct `ThemeData` configurations for `theme` (light) and `darkTheme`.
 - Use state management to toggle a `_themeMode` state variable (`ThemeMode.light` vs `ThemeMode.dark`).
 - Apply colors from `Theme.of(context)` inside descendant widgets.

// Experiment 6(b): App Themes & Dynamic Styling

```
import 'package:flutter/material.dart';
```

```
void main() => runApp(const DynamicThemeApp());
```

```
class DynamicThemeApp extends StatefulWidget {
```

```
  const DynamicThemeApp({super.key});
```

```
  @override
```

```
  State<DynamicThemeApp> createState() => _DynamicThemeAppState();
```

```
}
```

```

class _DynamicThemeAppState extends State<DynamicThemeApp> {
  ThemeMode _themeMode = ThemeMode.light;

  void _toggleTheme() {
    setState(() {
      _themeMode = _themeMode == ThemeMode.light ? ThemeMode.dark : ThemeMode.light;
    });
  }
}

```

```

@override
Widget build(BuildContext context) {
  return MaterialApp(
    debugShowCheckedModeBanner: false,
    title: 'App Themes',
    themeMode: _themeMode,
    theme: ThemeData(
      brightness: Brightness.light,
      primarySwatch: Colors.teal,
      scaffoldBackgroundColor: Colors.grey.shade100,
      cardColor: Colors.white,
    ),
    darkTheme: ThemeData(
      brightness: Brightness.dark,
      primarySwatch: Colors.teal,
      scaffoldBackgroundColor: Colors.grey.shade900,
      cardColor: Colors.grey.shade800,
    ),
    home: ThemeHomeScreen(
      onToggleTheme: _toggleTheme,
      currentMode: _themeMode,
    ),
  );
}

```

```

class ThemeHomeScreen extends StatelessWidget {
  final VoidCallback onToggleTheme;
  final ThemeMode currentMode;

  const ThemeHomeScreen({
    super.key,
    required this.onToggleTheme,
    required this.currentMode,
  });
}

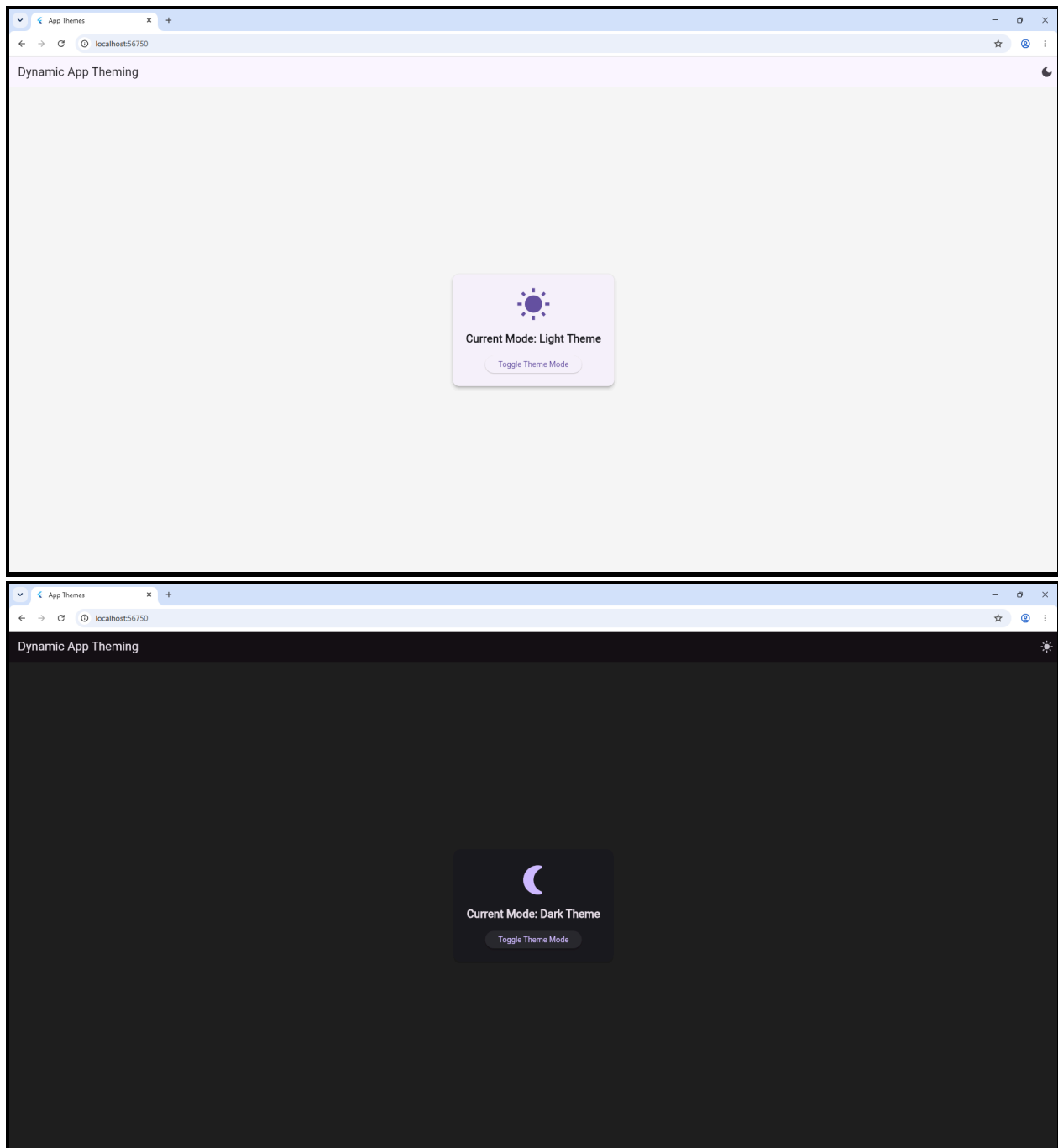
```

```

@override
Widget build(BuildContext context) {
  bool isDark = currentMode == ThemeMode.dark;

  return Scaffold(
    appBar: AppBar(
      title: const Text('Dynamic App Theming'),
      actions: [
        IconButton(
          icon: Icon(isDark ? Icons.light_mode : Icons.dark_mode),
          onPressed: onToggleTheme,
        ),
      ],
    ),
    body: Center(
      child: Card(
        elevation: 4,
        margin: const EdgeInsets.all(20),
        child: Padding(
          padding: const EdgeInsets.all(24.0),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.min,
            children: [
              Icon(
                isDark ? Icons.nightlight_round : Icons.wb_sunny,
                size: 64,
                color: Theme.of(context).colorScheme.primary,
              ),
              const SizedBox(height: 16),
              Text(
                'Current Mode: ${isDark ? "Dark Theme" : "Light Theme"}',
                style: const TextStyle(fontSize: 20, fontWeight: FontWeight.bold),
              ),
              const SizedBox(height: 16),
              ElevatedButton(
                onPressed: onToggleTheme,
                child: const Text('Toggle Theme Mode'),
              ),
            ],
          ),
        ),
      ),
    ),
  );
}

```



Experiment 7(a): Form Input Fields

Objective:

To capture user inputs using `Form` and multiple `TextFormField` components across various data fields.

Theory & Procedure:

1. **Theory:** `TextFormField` wraps a standard `TextField` inside a `Form` container, integrating with form state management, focus traversal, and input decorations.
2. **Procedure:**
 - Create a `StatefulWidget` containing multiple `TextEditingController` instances (Name, Email, Department).
 - Arrange `TextFormField` widgets with `InputDecoration` inside a `Column`.
 - Dispose of all controllers inside `dispose()` to prevent memory leaks.

// Experiment 7(a): Form Input Fields

```
import 'package:flutter/material.dart';
```

```
void main() => runApp(const FormInputApp());
```

```
class FormInputApp extends StatelessWidget {  
  const FormInputApp({super.key});
```

```
  @override  
  Widget build(BuildContext context) {  
    return const MaterialApp(  
      debugShowCheckedModeBanner: false,  
      home: StudentFormScreen(),  
    );  
  }  
}
```

```
class StudentFormScreen extends StatefulWidget {  
  const StudentFormScreen({super.key});
```

```
  @override  
  State<StudentFormScreen> createState() => _StudentFormScreenState();  
}
```

```
class _StudentFormScreenState extends State<StudentFormScreen> {  
  final TextEditingController _nameController = TextEditingController();  
  final TextEditingController _emailController = TextEditingController();  
  final TextEditingController _deptController = TextEditingController();
```

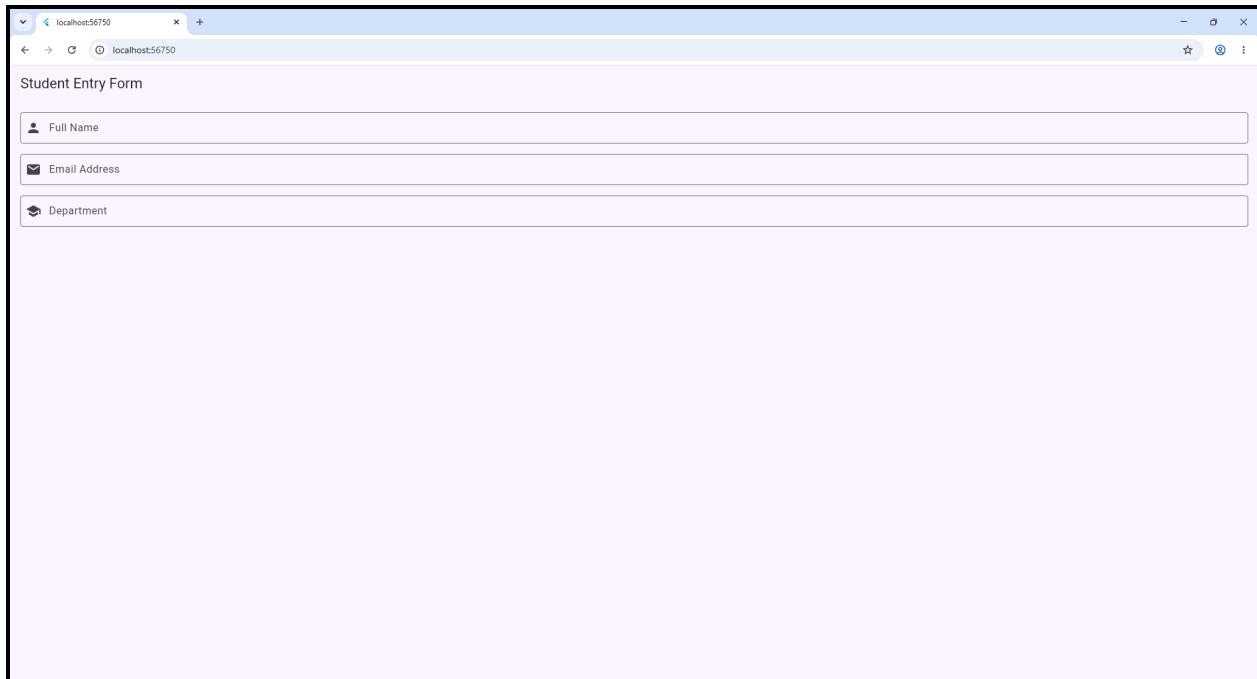
```
  @override  
  void dispose() {  
    _nameController.dispose();  
    _emailController.dispose();  
    _deptController.dispose();  
    super.dispose();  
  }
```

```
  @override  
  Widget build(BuildContext context) {
```

```

return Scaffold(
  appBar: AppBar(title: const Text('Student Entry Form')),
  body: Padding(
    padding: const EdgeInsets.all(16.0),
    child: SingleChildScrollView(
      child: Column(
        children: [
          TextFormField(
            controller: _nameController,
            decoration: const InputDecoration(
              labelText: 'Full Name',
              prefixIcon: Icon(Icons.person),
              border: OutlineInputBorder(),
            ),
          ),
          const SizedBox(height: 16),
          TextFormField(
            controller: _emailController,
            keyboardType: TextInputType.emailAddress,
            decoration: const InputDecoration(
              labelText: 'Email Address',
              prefixIcon: Icon(Icons.email),
              border: OutlineInputBorder(),
            ),
          ),
          const SizedBox(height: 16),
          TextFormField(
            controller: _deptController,
            decoration: const InputDecoration(
              labelText: 'Department',
              prefixIcon: Icon(Icons.school),
              border: OutlineInputBorder(),
            ),
          ),
        ],
      ),
    ),
  );
}

```



Experiment 7(b): Form Validation & Error Handling

Objective:

To validate user inputs using a `GlobalKey<FormState>` and display inline error messages and feedback snackbars.

Theory & Procedure:

1. **Theory:** A `GlobalKey<FormState>` uniquely identifies a `Form` and provides access to `FormState`. Calling `_formKey.currentState!.validate()` executes individual `validator` functions for all child `TextFormField` widgets.
2. **Procedure:**
 - Attach `_formKey` to the `Form` widget.
 - Provide `validator` callbacks to check for empty or malformed inputs.
 - If validation succeeds, display confirmation via `ScaffoldMessenger`.

// Experiment 7(b): Form Validation & Error Handling

```
import 'package:flutter/material.dart';
```

```
void main() => runApp(const FormValidationApp());
```

```
class FormValidationApp extends StatelessWidget {  
  const FormValidationApp({super.key});
```

```
  @override
```

```
  Widget build(BuildContext context) {
```

```
    return const MaterialApp(  
      debugShowCheckedModeBanner: false,
```

```

        home: ValidatedFormScreen(),
    );
}
}

```

```

class ValidatedFormScreen extends StatefulWidget {
  const ValidatedFormScreen({super.key});

```

```

  @override
  State<ValidatedFormScreen> createState() => _ValidatedFormScreenState();
}

```

```

class _ValidatedFormScreenState extends State<ValidatedFormScreen> {
  final formKey = GlobalKey<FormState>();
  final TextEditingController _emailController = TextEditingController();
  final TextEditingController _passwordController = TextEditingController();

```

```

  @override
  void dispose() {
    _emailController.dispose();
    _passwordController.dispose();
    super.dispose();
  }

```

```

  void _submitForm() {
    if (formKey.currentState!.validate()) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: Text('Form Validated! Logging in ${_emailController.text}'),
          backgroundColor: Colors.green,
        ),
      );
    }
  }
}

```

```

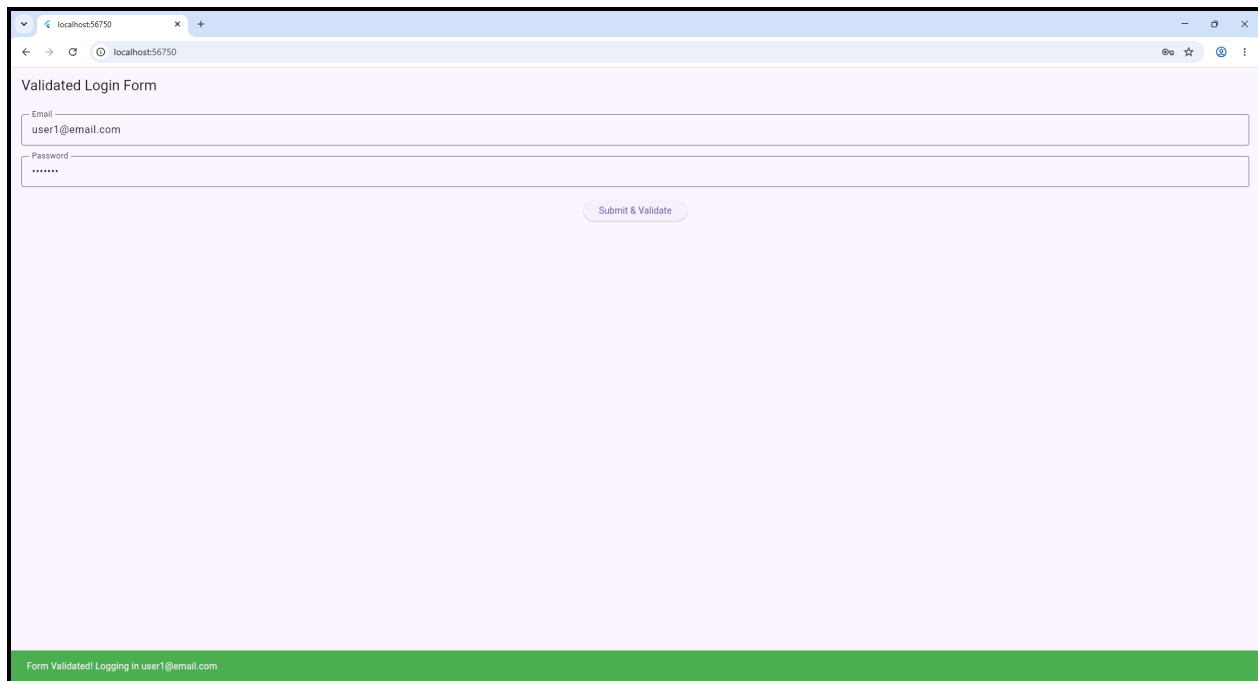
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Validated Login Form')),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Form(
          key: _formKey,
          child: Column(
            children: [
              TextFormField(

```

```

        controller: _emailController,
        decoration: const InputDecoration(
          labelText: 'Email',
          border: OutlineInputBorder(),
        ),
        validator: (value) {
          if (value == null || value.trim().isEmpty) {
            return 'Email cannot be empty';
          }
          if (!value.contains('@') || !value.contains('.')) {
            return 'Please enter a valid email address';
          }
          return null;
        },
      ),
      const SizedBox(height: 16),
      TextFormField(
        controller: _passwordController,
        obscureText: true,
        decoration: const InputDecoration(
          labelText: 'Password',
          border: OutlineInputBorder(),
        ),
        validator: (value) {
          if (value == null || value.length < 6) {
            return 'Password must be at least 6 characters';
          }
          return null;
        },
      ),
      const SizedBox(height: 20),
      ElevatedButton(
        onPressed: _submitForm,
        child: const Text('Submit & Validate'),
      ),
    ],
  ),
),
),
),
);
}
}

```



Experiment 8(a): Implicit Animations

Objective:

To build smooth property transitions using implicit animation widgets (`AnimatedContainer` and `AnimatedOpacity`).

Theory & Procedure:

1. **Theory:** Implicit animations manage their own `AnimationController` internally. Whenever widget properties (such as width, height, color, or opacity) change during a rebuild, the framework automatically animates the transition over a specified `duration`.
2. **Procedure:**
 - o Maintain state variables for size, color, and visibility flag.
 - o Wrap UI elements inside `AnimatedContainer` and `AnimatedOpacity`.
 - o Mutate values inside `setState()` on user interaction.

// Experiment 8(a): Implicit Animations

```
import 'package:flutter/material.dart';
```

```
void main() => runApp(const ImplicitAnimationApp());
```

```
class ImplicitAnimationApp extends StatelessWidget {  
  const ImplicitAnimationApp({super.key});
```

```
  @override  
  Widget build(BuildContext context) {  
    return const MaterialApp(  
      debugShowCheckedModeBanner: false,
```

```

    home: ImplicitAnimationScreen(),
  );
}
}

```

```

class ImplicitAnimationScreen extends StatefulWidget {
  const ImplicitAnimationScreen({super.key});

```

```

  @override
  State<ImplicitAnimationScreen> createState() => _ImplicitAnimationScreenState();
}

```

```

class _ImplicitAnimationScreenState extends State<ImplicitAnimationScreen> {
  bool _isExpanded = false;
  bool _isVisible = true;

```

```

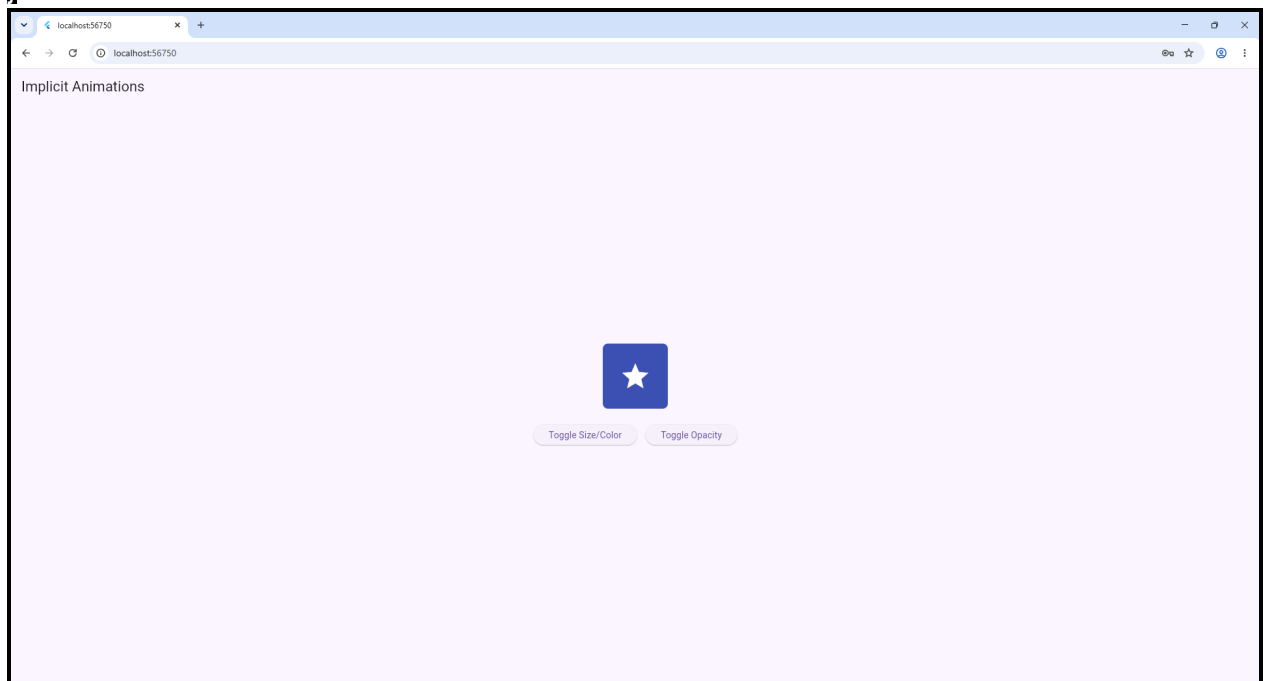
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Implicit Animations')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            AnimatedOpacity(
              duration: const Duration(milliseconds: 600),
              opacity: _isVisible ? 1.0 : 0.0,
              child: AnimatedContainer(
                duration: const Duration(milliseconds: 600),
                curve: Curves.easeInOut,
                width: _isExpanded ? 200.0 : 100.0,
                height: _isExpanded ? 200.0 : 100.0,
                decoration: BoxDecoration(
                  color: _isExpanded ? Colors.deepOrange : Colors.indigo,
                  borderRadius: BorderRadius.circular(_isExpanded ? 30 : 8),
                ),
                child: const Icon(Icons.star, color: Colors.white, size: 48),
              ),
            ),
            const SizedBox(height: 24),
            Row(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                ElevatedButton(
                  onPressed: () => setState(() => _isExpanded = !_isExpanded),
                  child: const Text('Toggle Size/Color'),
                ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}

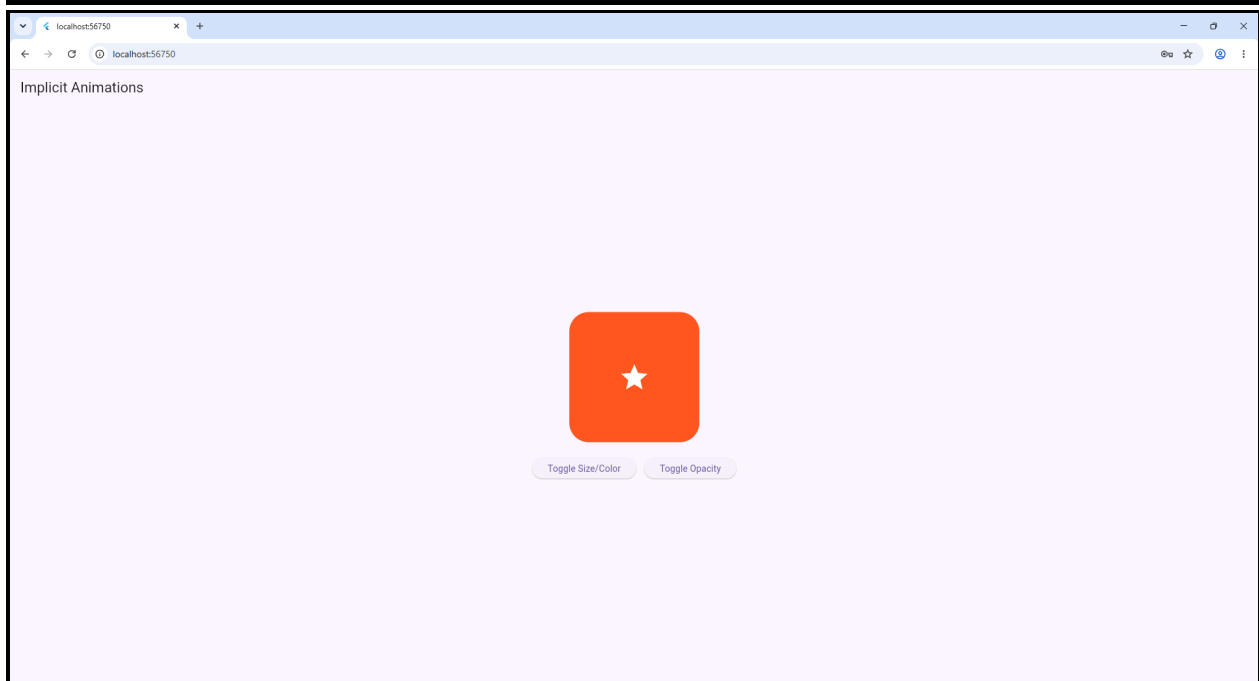
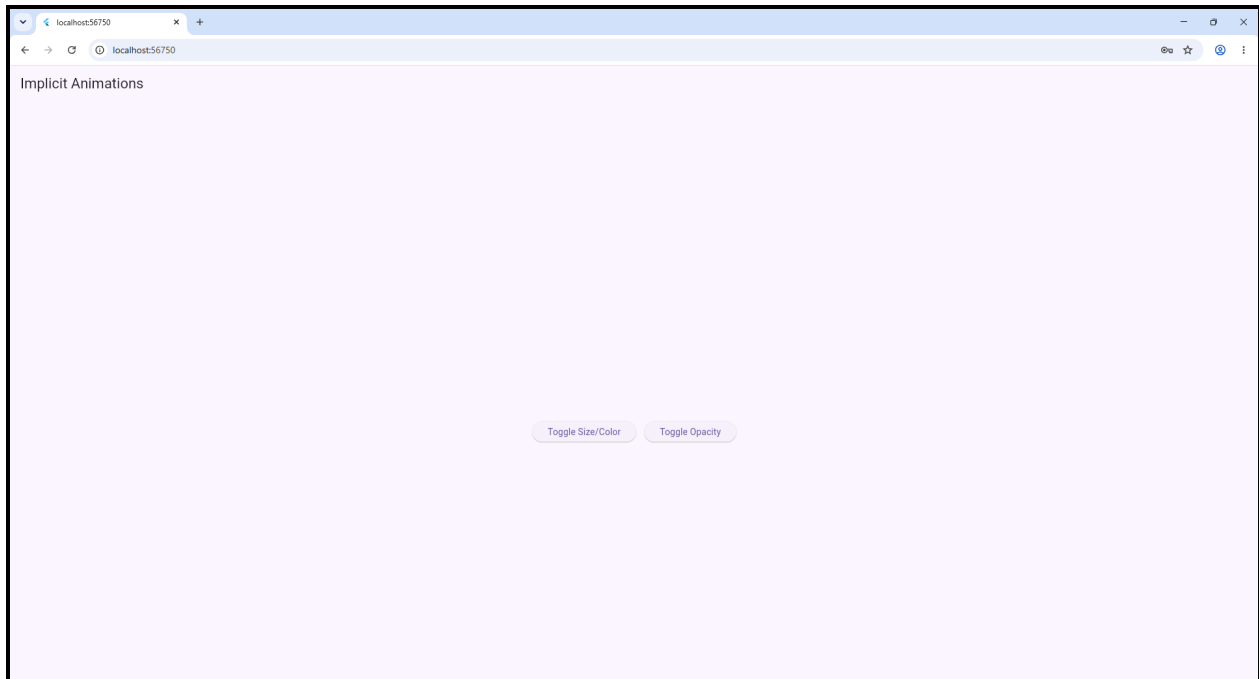
```

```

    ),
    const SizedBox(width: 12),
    ElevatedButton(
      onPressed: () => setState(() => _isVisible = !_isVisible),
      child: const Text('Toggle Opacity'),
    ),
  ],
),
],
),
],
),
),
),
),
),
},
}
}

```





Experiment 8(b): Explicit Composite Animations (Fade + Slide)

Objective:

To construct explicit, synchronized animations combining `FadeTransition` and `SlideTransition` using `AnimationController`.

Theory & Procedure:

1. **Theory:** Explicit animations offer complete control over playback, repetition, direction, and curves using an `AnimationController` and `TickerProviderStateMixin`.
2. **Procedure:**
 - Mix `SingleTickerProviderStateMixin` into the `State` class.
 - Initialize `AnimationController`, `_fadeAnimation`, and `_slideAnimation` inside `initState()`.
 - Wrap the child widget inside nested `FadeTransition` and `SlideTransition` widgets.
 - Dispose of the controller inside `dispose()`.

// Experiment 8(b): Explicit Composite Animations

```
import 'package:flutter/material.dart';
```

```
void main() => runApp(const ExplicitAnimationApp());
```

```
class ExplicitAnimationApp extends StatelessWidget {  
  const ExplicitAnimationApp({super.key});
```

```
  @override  
  Widget build(BuildContext context) {  
    return const MaterialApp(  
      debugShowCheckedModeBanner: false,  
      home: CompositeAnimationScreen(),  
    );  
  }  
}
```

```
class CompositeAnimationScreen extends StatefulWidget {  
  const CompositeAnimationScreen({super.key});
```

```
  @override  
  State<CompositeAnimationScreen> createState() => _CompositeAnimationScreenState();  
}
```

```
class _CompositeAnimationScreenState extends State<CompositeAnimationScreen>  
  with SingleTickerProviderStateMixin {  
  late final AnimationController _controller;  
  late final Animation<double> _fadeAnimation;  
  late final Animation<Offset> _slideAnimation;
```

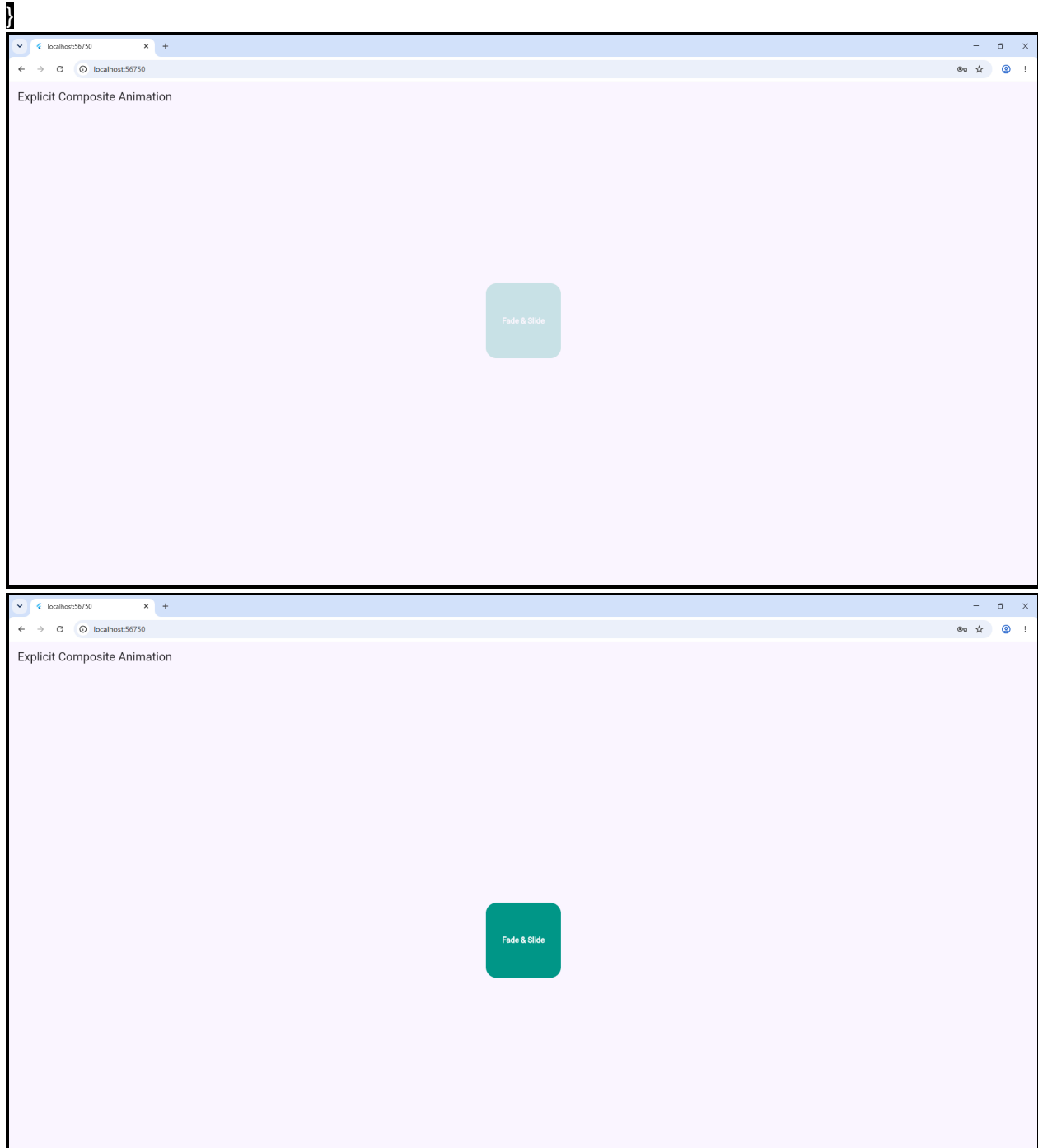
```
  @override  
  void initState() {  
    super.initState();  
    _controller = AnimationController(  
      vsync: this,  
      duration: const Duration(seconds: 2),  
    )..repeat(reverse: true);
```

```
_fadeAnimation = Tween<double>(begin: 0.2, end: 1.0).animate(  
  CurvedAnimation(parent: _controller, curve: Curves.easeInOut),  
);
```

```
_slideAnimation = Tween<Offset>(  
  begin: const Offset(0.0, -0.4),  
  end: const Offset(0.0, 0.4),  
).animate(  
  CurvedAnimation(parent: _controller, curve: Curves.easeInOut),  
);  
}
```

```
@override  
void dispose() {  
  _controller.dispose();  
  super.dispose();  
}
```

```
@override  
Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(title: const Text('Explicit Composite Animation')),  
    body: Center(  
      child: SlideTransition(  
        position: _slideAnimation,  
        child: FadeTransition(  
          opacity: _fadeAnimation,  
          child: Container(  
            width: 140,  
            height: 140,  
            decoration: BoxDecoration(  
              color: Colors.teal,  
              borderRadius: BorderRadius.circular(20),  
            ),  
            child: const Center(  
              child: Text(  
                'Fade & Slide',  
                style: TextStyle(color: Colors.white, fontWeight: FontWeight.bold),  
              ),  
            ),  
          ),  
        ),  
      ),  
    ),  
  );  
}
```



Experiment 9(a): Asynchronous REST API Fetching

Objective:

To issue asynchronous HTTP GET requests using the `http` package and decode remote JSON responses.

Theory & Procedure: add `http` package by using `flutter pub add http` command

1. **Theory:** Mobile applications communicate with backend services over HTTP/HTTPS. The `http` package executes non-blocking requests, returning `Future<http.Response>` payloads that are parsed using `jsonDecode()`.
2. **Procedure:**
 - Issue an `http.get(Uri.parse(...))` call to a REST endpoint (e.g., JSONPlaceholder).
 - Verify status code `200 OK`.
 - Decode payload string into a `List<dynamic>` using `jsonDecode()`.

// Experiment 9(a): Asynchronous REST API Fetching

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
```

```
void main() => runApp(const ApiFetchApp());
```

```
class ApiFetchApp extends StatelessWidget {
  const ApiFetchApp({super.key});
```

```
  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: UserListApiScreen(),
    );
  }
}
```

```
class UserListApiScreen extends StatefulWidget {
  const UserListApiScreen({super.key});
```

```
  @override
  State<UserListApiScreen> createState() => _UserListApiScreenState();
}
```

```
class _UserListApiScreenState extends State<UserListApiScreen> {
  List<dynamic> _users = [];
  bool _isLoading = true;
  String? _error;
```

```
  @override
  void initState() {
    super.initState();
    _fetchUsers();
  }
```

```
  Future<void> _fetchUsers() async {
    try {
```

```

    final response = await http.get(
      Uri.parse('https://jsonplaceholder.typicode.com/users'),
    );

```

```

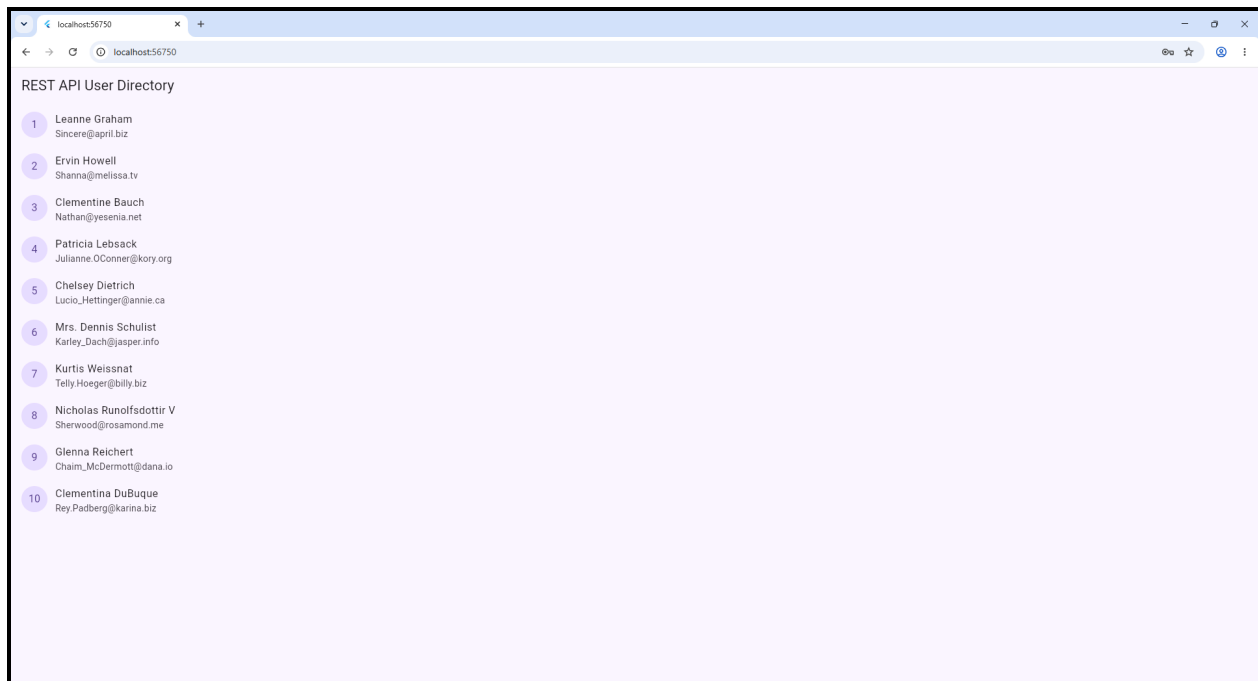
    if (response.statusCode == 200) {
      setState(() {
        _users = jsonDecode(response.body);
        _isLoading = false;
      });
    } else {
      setState(() {
        _error = 'Failed code: ${response.statusCode}';
        _isLoading = false;
      });
    }
  } catch (e) {
    setState(() {
      _error = 'Connection exception: $e';
      _isLoading = false;
    });
  }
}

```

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('REST API User Directory')),
    body: _isLoading
      ? const Center(child: CircularProgressIndicator())
      : _error != null
        ? Center(child: Text(_error!))
        : ListView.builder(
            itemCount: _users.length,
            itemBuilder: (context, index) {
              final user = _users[index];
              return ListTile(
                leading: CircleAvatar(child: Text('${user["id"]}')),
                title: Text(user['name']),
                subtitle: Text(user['email']),
              );
            },
          ),
  );
}

```



Experiment 9(b): Displaying Fetched Data Meaningfully with FutureBuilder

Objective:

To render asynchronous REST API data using **FutureBuilder** while initializing the **Future** in **initState()** to prevent redundant re-fetching on widget rebuilds.

Theory & Procedure:

1. **Theory:** **FutureBuilder** subscribes to a **Future** and builds UI based on snapshot connection states (**ConnectionState.waiting**, **hasError**, **hasData**). Caching the **Future** in **initState()** ensures network requests execute exactly once per widget lifecycle.
2. **Procedure:**
 - Declare **late final Future<List<dynamic>> _postsFuture;**
 - Initialize **_postsFuture = _fetchPosts();** inside **initState()**.
 - Pass **_postsFuture** to **FutureBuilder** and render list items on success.

// Experiment 9(b): FutureBuilder REST API Integration

```
import 'dart:convert';
import 'package:flutter/material.dart';
import 'package:http/http.dart' as http;
```

```
void main() => runApp(const FutureBuilderApp());
```

```
class FutureBuilderApp extends StatelessWidget {
  const FutureBuilderApp({super.key});
```

```
  @override
  Widget build(BuildContext context) {
```

```

return const MaterialApp(
  debugShowCheckedModeBanner: false,
  home: PostListScreen(),
);
}
}

```

```

class PostListScreen extends StatefulWidget {
  const PostListScreen({super.key});

```

```

  @override
  State<PostListScreen> createState() => _PostListScreenState();
}

```

```

class _PostListScreenState extends State<PostListScreen> {
  // Cached Future instance to prevent refetching during rebuilds
  late final Future<List<dynamic>> _postsFuture;

```

```

  @override
  void initState() {
    super.initState();
    _postsFuture = _fetchPosts();
  }

```

```

  Future<List<dynamic>> _fetchPosts() async {
    final response = await http.get(
      Uri.parse('https://jsonplaceholder.typicode.com/posts?_limit=5'),
    );

```

```

    if (response.statusCode == 200) {
      return jsonDecode(response.body) as List<dynamic>;
    } else {
      throw Exception('Failed to load posts (Code: ${response.statusCode})');
    }
  }
}

```

```

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('FutureBuilder Posts View')),
      body: FutureBuilder<List<dynamic>>(
        future: _postsFuture,
        builder: (context, snapshot) {
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Center(child: CircularProgressIndicator());
          } else if (snapshot.hasError) {

```

A screenshot of a web browser window. The address bar at the top shows 'localhost:56750'. The page title is 'FutureBuilder Posts View'. The page content consists of four posts, each in a light purple box. Each post has a title in bold and a paragraph of Latin text. The posts are: 1. Title: 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit'. Text: 'quia et suscipit', 'suscipit recusandae consequuntur expedita et cum', 'reprehenderit molestiae ut ut quas totam', 'nostrum rerum est autem sunt rem eveniet architecto'. 2. Title: 'qui est esse'. Text: 'est rerum tempore vitae', 'sequi sint nihil reprehenderit dolor beatae ea dolores neque', 'fugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis', 'qui aperiam non debitis possimus qui neque nisi nulla'. 3. Title: 'ea molestias quasi exercitationem repellat qui ipsa sit aut'. Text: 'et iusto sed quo iure', 'voluptatem occaecati omnis eligendi aut ad', 'voluptatem doloribus vel accusantium quis pariatur', 'molestiae porro eius odio et labore et velit aut'. 4. Title: 'eum et est occaecati'. Text: 'ullam et saepe reiciendis voluptatem adipisci', 'sit amet autem assumenda provident rerum culpa', 'quis hic commodi nesciunt rem tenetur doloremque ipsam iure', 'quis sunt voluptatem rerum illo velit'. 5. Title: 'nesciunt quas odio'. Text: 'repudiandae veniam quaerat sunt sed', 'alias aut fugiat sit autem sed est', 'voluptatem omnis possimus esse voluptatibus quis', 'est aut tenetur dolor neque'.

Objective:

To construct pure Dart function unit tests and UI component widget tests using the `flutter_test` framework.

Theory & Procedure:

1. **Theory:** Automated tests verify software correctness. Unit tests validate pure logic using `test()` and `expect()`. Widget tests render components in a headless test environment using `testWidgets()`, `pumpWidget()`, `find.text()`, and `findsOneWidget`.
2. **Procedure:** use command `flutter test`
 - Write a standalone unit test checking arithmetic logic.
 - Write a widget test that renders a target widget hierarchy via `tester.pumpWidget()` and verifies rendered elements.

The following code has to be in `widget_test.dart`

```
// Experiment 10(a): Automated Unit & Widget Testing
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';

// 1. Pure Function for Unit Testing
int calculateTotalMarks(int sub1, int sub2) => sub1 + sub2;

void main() {
  // Group 1: Unit Tests
  group('Pure Logic Unit Tests', () {
    test('calculateTotalMarks computes correct sum', () {
      int result = calculateTotalMarks(45, 50);
      expect(result, 95);
    });
  });

  // Group 2: Widget Tests
  group('UI Component Widget Tests', () {
    testWidgets('Verify Header Title Renders Correctly', (WidgetTester tester) async {
      await tester.pumpWidget(
        const MaterialApp(
          home: Scaffold(
            body: Text('Campus Companion Portal'),
          ),
        ),
      );
    });

    // Assert text presence using finder and matcher
    expect(find.text('Campus Companion Portal'), findsOneWidget);
  });
}
```

Experiment 10(b): DevTools, Logging, and Debugging Workflow

Objective:

To log diagnostic output using `developer.log()` and implement an interactive debugging workflow to identify and resolve logic defects.

Theory & Procedure:

1. **Theory:** Diagnostic logging using `developer.log()` sends structured logs to Flutter DevTools without hitting console print throttling.
2. **Procedure:**
 - Create a price calculator widget with a deliberate logic bug.
 - Log buggy outputs via `developer.log()` to inspect behavior.
 - Apply corrected calculations, log clean diagnostic output, and update screen state.

// Experiment 10(b): DevTools Logging & Debugging Workflow

```
import 'dart:developer' as developer;
import 'package:flutter/material.dart';
```

```
void main() => runApp(const DebuggingApp());
```

```
class DebuggingApp extends StatelessWidget {
  const DebuggingApp({super.key});
```

```
  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: DebuggingCalculatorScreen(),
    );
  }
}
```

```
class DebuggingCalculatorScreen extends StatefulWidget {
  const DebuggingCalculatorScreen({super.key});
```

```
  @override
  State<DebuggingCalculatorScreen> createState() => _DebuggingCalculatorScreenState();
}
```

```
class _DebuggingCalculatorScreenState extends State<DebuggingCalculatorScreen> {
  final double _price = 100.0;
  final double _taxRate = 0.15; // 15% tax
  double? _buggyResult;
  double? _fixedResult;
```

```
  void _executeDiagnosticRun() {
```

```

// 1. Deliberate logic bug: tax is incorrectly neutralized by multiplying by 0.0
_buggyResult = _price + (_price * _taxRate * 0.0);
developer.log(
  'DEBUG [Buggy Calc]: Base=$_price, Output=$_buggyResult (Expected: \$115.0)',
  name: 'scet.lab.exp10b',
  level: 900, // Warning level
);

// 2. Corrected tax calculation
_fixedResult = _price + (_price * _taxRate);
developer.log(
  'DEBUG [Fixed Calc]: Base=$_price, Tax=${_taxRate * 100}%, Output=$_fixedResult',
  name: 'scet.lab.exp10b',
  level: 500, // Info level
);

setState(() {});
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('DevTools Logging & Debugging')),
    body: Padding(
      padding: const EdgeInsets.all(16.0),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            'Base Price: \${$_price.toStringAsFixed(2)} | Tax Rate: 15%',
            style: const TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
          ),
          const SizedBox(height: 16),
          ElevatedButton(
            onPressed: _executeDiagnosticRun,
            child: const Text('Execute & Log Debug Run'),
          ),
          const SizedBox(height: 24),
          if (_buggyResult != null) ...[
            Text(
              'Buggy Output (Check Console): \${_buggyResult!.toStringAsFixed(2)}',
              style: const TextStyle(color: Colors.red, fontSize: 16),
            ),
            const SizedBox(height: 8),
            Text(
              'Corrected Output: \${_fixedResult!.toStringAsFixed(2)}',

```

```
style: const TextStyle(color: Colors.green, fontSize: 16, fontWeight: FontWeight.bold),  
),  
],  
],  
),  
),  
);  
}  
}
```